

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ  
ОБРАЗОВАТЕЛЬНОЕ  
УЧРЕЖДЕНИЕ ВЫСШЕГО ПРОФЕССИОНАЛЬНОГО ОБРАЗОВАНИЯ  
«МОСКОВСКИЙ АВИАЦИОННЫЙ ИНСТИТУТ  
(национальный исследовательский университет)» (МАИ)

---

Направление подготовки: 27.03.05 «Инноватика»

**Лабораторная работа №2**

по дисциплине «Структура программного обеспечения и базы данных»

Выполнили:

Студенты гр. МЗО-236БВ-24

Никандров И.А.

Зайцев Д.Ю.

Карамян Ф.Э.

Преподаватель:

Александрова С.С.

Прудников А.В.

Москва 2025

## **Оглавление**

|                                      |    |
|--------------------------------------|----|
| Задание на лабораторную работу ..... | 3  |
| Ход выполнения работы .....          | 3  |
| Глава 1 .....                        | 3  |
| Глава 2 .....                        | 20 |
| Вывод: .....                         | 38 |
| Ссылка на репозиторий .....          | 39 |

## Задание на лабораторную работу

Цель работы: зайти в админку джанго на локалхосте

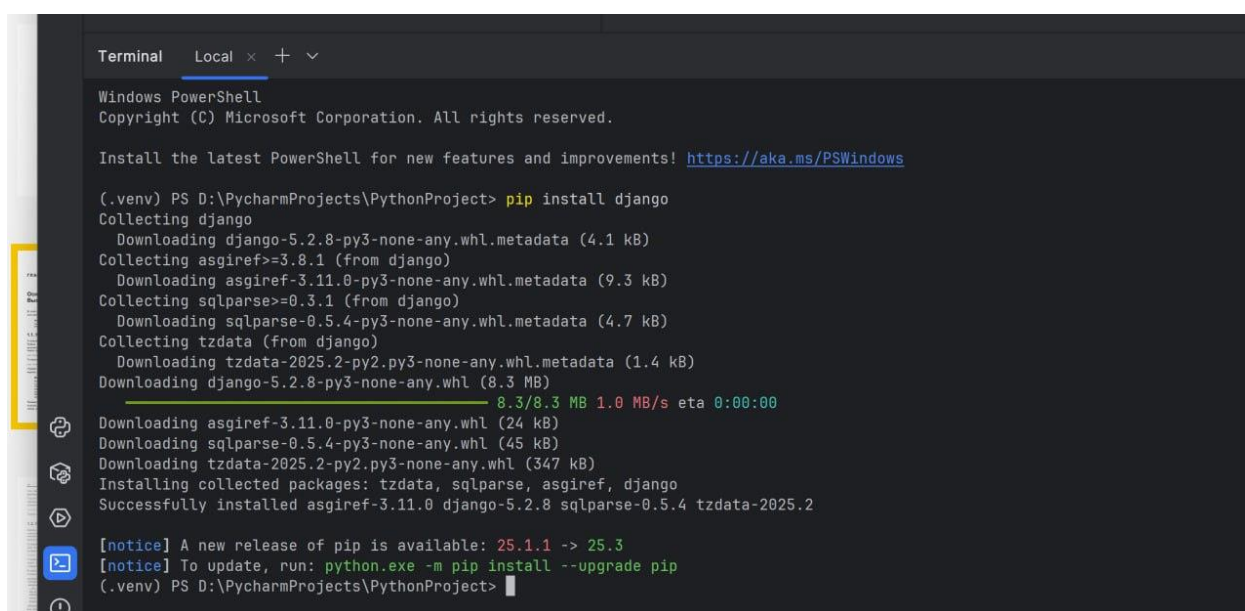
Задачи работы: изучить [учебник](#) по Django выполнить 1-2 главу, залить на гит, составить отчет

### Ход выполнения работы

Выполнить 1-2 главу

#### Глава 1

##### 1.1 Устанавливаем фреймворк



```
Terminal Local x + v
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

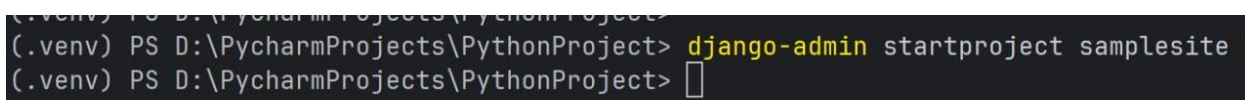
Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows

(.venv) PS D:\PycharmProjects\PythonProject> pip install django
Collecting django
  Downloading django-5.2.8-py3-none-any.whl.metadata (4.1 kB)
Collecting asgiref<=3.8.1 (from django)
  Downloading asgiref-3.11.0-py3-none-any.whl.metadata (9.3 kB)
Collecting sqlparse>=0.3.1 (from django)
  Downloading sqlparse-0.5.4-py3-none-any.whl.metadata (4.7 kB)
Collecting tzdata (from django)
  Downloading tzdata-2025.2-py2.py3-none-any.whl.metadata (1.4 kB)
  Downloading django-5.2.8-py3-none-any.whl (8.3 MB)
    8.3/8.3 MB 1.0 MB/s eta 0:00:00
  Downloading asgiref-3.11.0-py3-none-any.whl (24 kB)
  Downloading sqlparse-0.5.4-py3-none-any.whl (45 kB)
  Downloading tzdata-2025.2-py2.py3-none-any.whl (347 kB)
Installing collected packages: tzdata, sqlparse, asgiref, django
Successfully installed asgiref-3.11.0 django-5.2.8 sqlparse-0.5.4 tzdata-2025.2

[notice] A new release of pip is available: 25.1.1 -> 25.3
[notice] To update, run: python.exe -m pip install --upgrade pip
(.venv) PS D:\PycharmProjects\PythonProject>
```

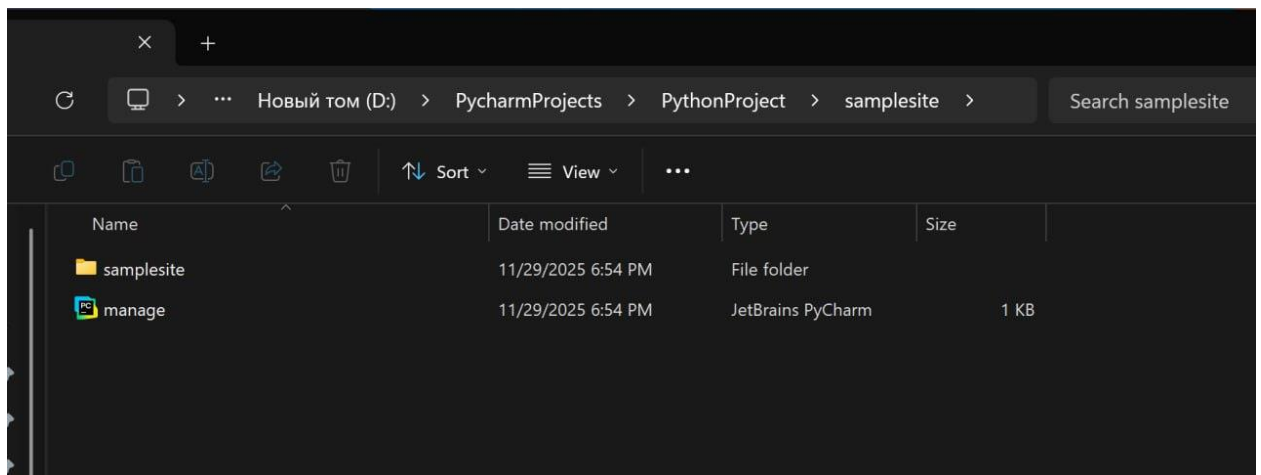
##### 1.2 Создаем проект Django

Создали новый, пока еще пустой проект Django которому дали имя samplesite. Для этого в запущенной ранее командной строке перешли в папку, в которой должна находиться папка проекта, и отдали команду: `django-admin startproject samplesite`

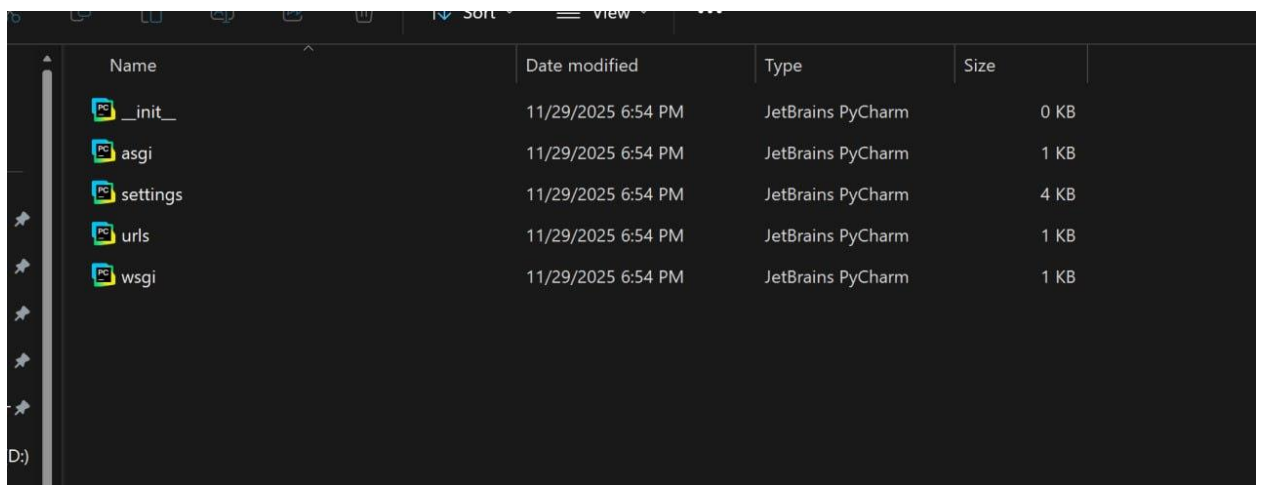


```
(.venv) PS D:\PycharmProjects\PythonProject> django-admin startproject samplesite
(.venv) PS D:\PycharmProjects\PythonProject>
```

Содержимое папки samplesite:



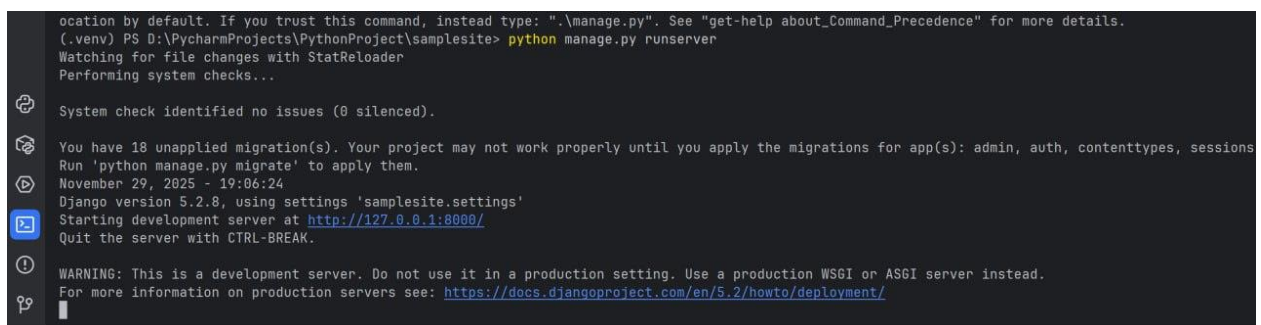
Пакет конфигурации включает в себя такие модули:



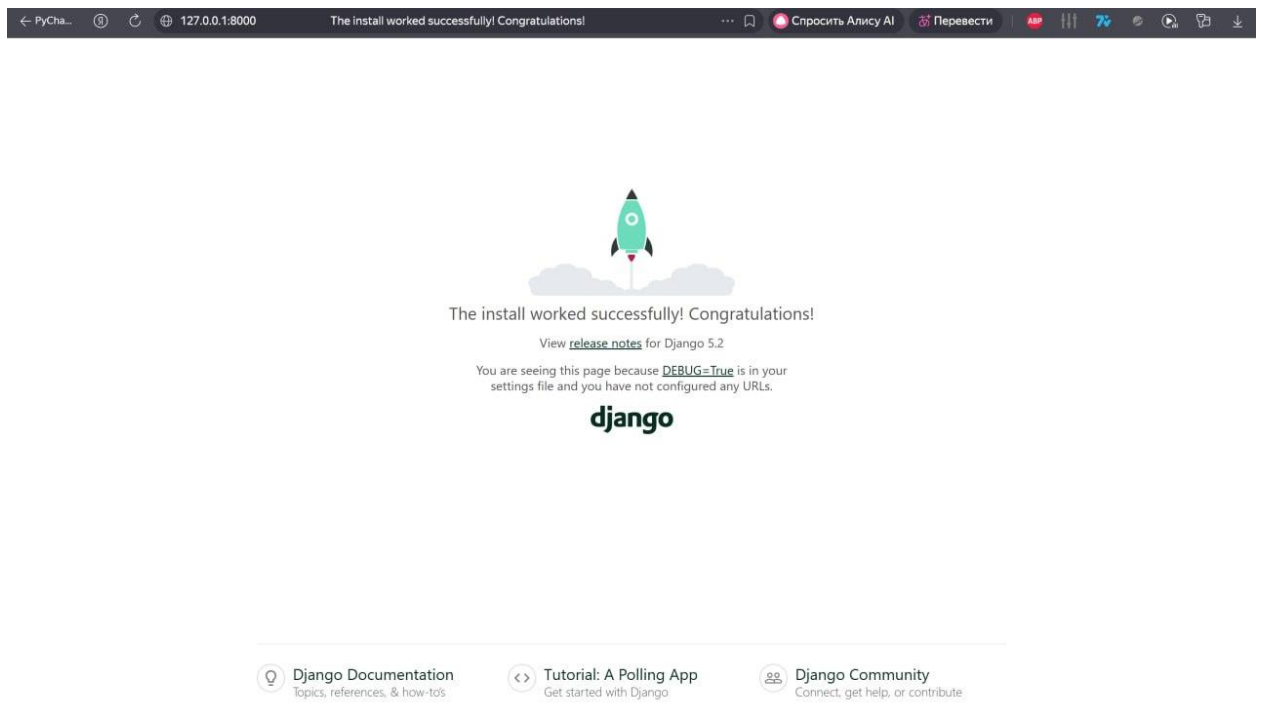
### 1.3. Отладочный веб-сервер Django

В состав Django входит отладочный веб-сервер, написанный на самом языке Python.

Сервер сообщил, что сайт успешно запущен



Информационная веб-страница Django, сообщающая о работоспособности вновь созданного «пустого» веб-сайта:



## 1.4. Приложения

### Прописываем в терминале

```
Suggestion [3,General]: The command manage.py was not found, but does exist in the current location. Window
ocation by default. If you trust this command, instead type: ".\manage.py". See "get-help about_Command_Pre
(.venv) PS D:\PycharmProjects\PythonProject\samplesite> python manage.py startapp bboard
(.venv) PS D:\PycharmProjects\PythonProject\samplesite> 
```

### Результат

| Name                                                                                           | Date modified      | Type              | Size |
|------------------------------------------------------------------------------------------------|--------------------|-------------------|------|
|  manage     | 11/29/2025 6:54 PM | JetBrains PyCharm | 1 KB |
|  db.sqlite3 | 11/29/2025 7:06 PM | SQLite3 File      | 0 KB |
|  samplesite | 11/29/2025 7:06 PM | File folder       |      |
|  bboard     | 11/29/2025 7:13 PM | File folder       |      |

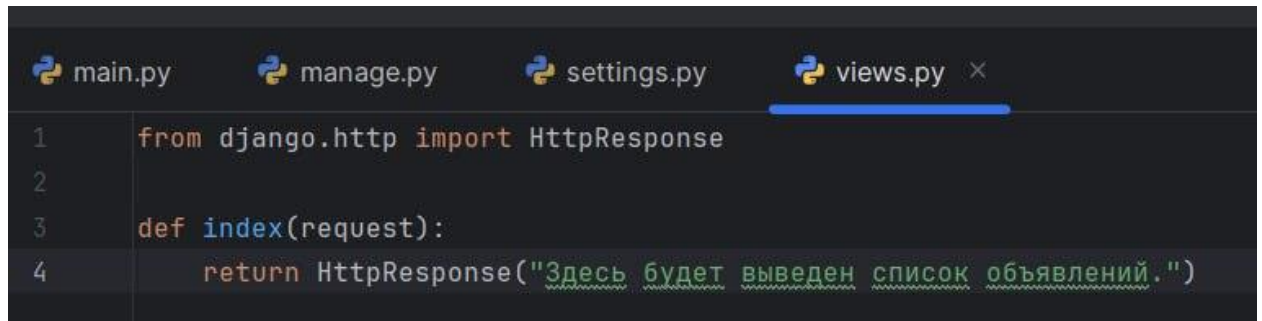
Регистрируем только что созданное приложение bboard

```
INSTALLED_APPS = [  
    'bboard'  
    'django.contrib.admin',  
    'django.contrib.auth',  
    'django.contrib.contenttypes',  
    'django.contrib.sessions',  
    'django.contrib.messages',  
    'django.contrib.staticfiles',  
]
```

### 1.5. Контроллеры

Написали контроллер-функцию, которая будет выводить текст, сообщающая, что будущие посетители сайта со временем увидят на этой странице список объявлений.

Для этого открыли модуль `views.py` пакета приложения `bboard`, удалили имеющийся там небольшой код и заменили его кодом из листинга 1.1



```
main.py  manage.py  settings.py  views.py ×  
1  from django.http import HttpResponse  
2  
3  def index(request):  
4      return HttpResponse("Здесь будет выведен список объявлений.")
```

### 1.6. Маршруты и маршрутизатор

В пакете конфигурации хранится модуль `urls.py`, в котором записываются маршруты уровня проекта.

Открыли этот модуль в текстовом редакторе и посмотрели, что он содержит:

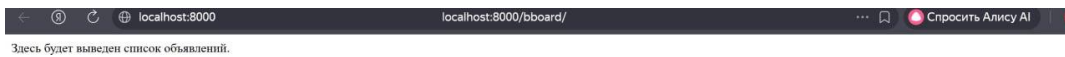
```

1  from django.contrib import admin
2  from django.urls import path
3  from bboard.views import index
4
5  urlpatterns = [
6      path(route: 'bboard/', index),
7      path(route: 'admin/', admin.site.urls),
8  ]

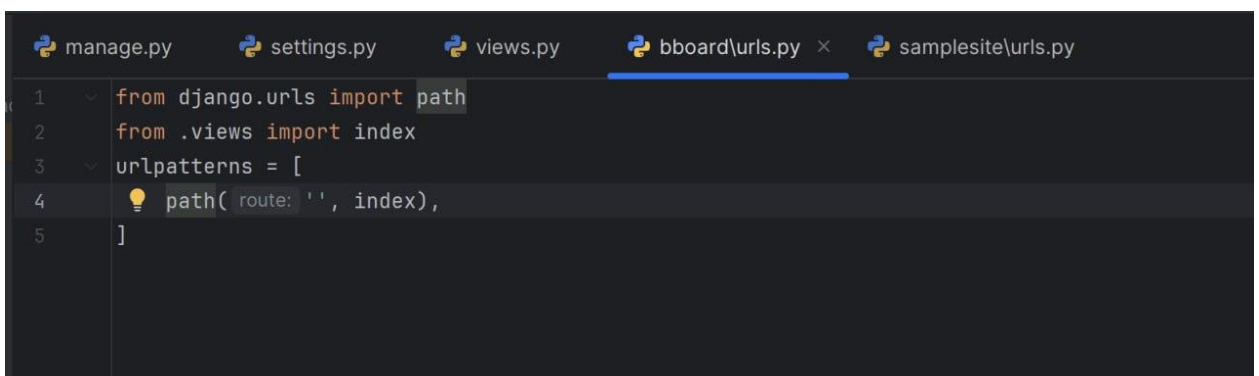
```

Сохранили исправленный модуль, запустили отладочный веб-сервер и набрали в веб обозревателе необходимый интернет-адрес.

После чего мы увидели текстовое сообщение, сгенерированное нашим контроллером:



Создадим в пакете этого приложения модуль urls.py и занесем в него код из листинга 1.4.



```

1  from django.urls import path
2  from .views import index
3  urlpatterns = [
4      path(route: '', index),
5  ]

```

Исправим код модуля urls.py из пакета конфигурации, как показано в листинге 1.5.

```
manage.py settings.py views.py bboard\urls.py samplesite\urls.py x
1 from django.contrib import admin
2 from django.urls import path, include
3 urlpatterns = [
4     path(route: 'bboard/', include('bboard.urls')),
5     path(route: 'admin/', admin.site.urls),
6 ]
```

## 1.7. Модели

Сделали так, чтобы вместо текстового сообщения выводились реальные объявления, хранящиеся в таблице базы данных.

Для этого мы прежде всего объявили модель Bb, представляющую объявление. (Все поля сделали обязательными для заполнения).

И завершили работу отладочного веб-сервера и открыли модуль models.py пакета приложения bboard и записали в него код из листинга 1.6.

```
manage.py settings.py views.py bboard\urls.py samplesite\urls.py models.py x
1 from django.db import models
2 class Bb(models.Model):
3     title = models.CharField(max_length=50)
4     content = models.TextField()
5     price = models.FloatField()
6     published = models.DateTimeField(auto_now_add=True, db_index=True)
```

## 1.8. Миграции

Чтобы сгенерировать нашу первую миграцию, проверим, остановлен ли отладочный веб-сервер и находимся ли мы в папке проекта, и дадим команду:

```
[29/Nov/2025 19:54:12] "GET /admin/login/?next=/admin/ HTTP/1.1" 200 4173
(.venv) PS D:\PycharmProjects\PythonProject\samplesite> python manage.py makemigrations bboard
Migrations for 'bboard':
  bboard\migrations\0001_initial.py
    + Create model Bb
(.venv) PS D:\PycharmProjects\PythonProject\samplesite> |
```

После команды sqlmigrate, выводящей SQL-код, мы поставили имя приложения и числовую часть имени модуля с миграцией. Прямо в командной строке мы получим такой результат (для удобства чтения был переформатирован)



```
(.venv) PS D:\PycharmProjects\PythonProject\samplesite> python manage.py sqlmigrate bboard 0001
BEGIN;
--
-- Create model Bb
--
CREATE TABLE "bboard_bb" ("id" integer NOT NULL PRIMARY KEY AUTOINCREMENT, "title" varchar(50) NOT NULL, "content" text NOT NULL, "price" real NOT NULL, "published" datetime NOT NULL);
CREATE INDEX "bboard_bb_published_58fde1b5" ON "bboard_bb" ("published");
COMMIT;
(.venv) PS D:\PycharmProjects\PythonProject\samplesite>
```

Выполнили нашу первую миграцию. Для этого набрали в командной строке команду:

```
(.venv) PS C:\Users\jenyl\PycharmProjects\PythonProject4\django_main\samplesite> python manage.py migrate
Operations to perform:
  Apply all migrations: admin, auth, bboard, contenttypes, sessions
Running migrations:
  Applying contenttypes.0001_initial... OK
  Applying auth.0001_initial... OK
  Applying admin.0001_initial... OK
  Applying admin.0002_logentry_remove_auto_add... OK
  Applying admin.0003_logentry_add_action_flag_choices... OK
  Applying contenttypes.0002_remove_content_type_name... OK
  Applying auth.0002_alter_permission_name_max_length... OK
  Applying auth.0003_alter_user_email_max_length... OK
  Applying auth.0004_alter_user_username_opts... OK
  Applying auth.0005_alter_user_last_login_null... OK
  Applying auth.0006_require_contenttypes_0002... OK
  Applying auth.0007_alter_validators_add_error_messages... OK
  Applying auth.0008_alter_user_username_max_length... OK
  Applying auth.0009_alter_user_last_name_max_length... OK
  Applying auth.0010_alter_group_name_max_length... OK
  Applying auth.0011_update_proxy_permissions... OK
  Applying auth.0012_alter_user_first_name_max_length... OK
  Applying bboard.0001_initial... OK
```

## 1.9. Консоль Django

У нас есть готовая модель для хранения объявлений, но нет ни одного объявления. Создали парочку для целей отладки.

В командной строке набрали команду для запуска консоли Django, и сразу увидим знакомое приглашение >>>, предлагающее нам ввести какое-либо выражение Python и получить результат его выполнения:

```
(.venv) PS D:\PycharmProjects\PythonProject\samplesite> python manage.py shell
Terminal Alt+F12 ted automatically (use -v 2 for details).

Python 3.14.0 (tags/v3.14.0:ebf955d, Oct 7 2025, 10:15:03) [MSC v.1944 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license" for more information.
(InteractiveConsole)
>>> █
```

## 1.10. Работа с моделями

Проверим, сохранилось ли наше первое объявление, получив значение его ключевого поля:

```
(InteractiveConsole)
>>> from bboard.models import Bb
>>> b1 = Bb(title='Дача', content='Общество "Двухэтажники". ' + \
... 'Два этажа, кирпич, свет, газ, канализация', price=500000)
>>> b1.save()
>>> b1.pk
1
>>>
```

Мы можем обратиться к любому полю записи, воспользовавшись соответствующим ему атрибутом объекта модели:

```
>>> b1.pk
1
>>> b1.title
'Дача'
>>> b1.content
'Общество "Двухэтажники". Два этажа, кирпич, свет, газ, канализация'
>>> b1.price
500000
>>> b1.published
datetime.datetime(2025, 11, 29, 17, 58, 36, 622660, tzinfo=datetime.timezone.utc)
>>> b1.id
1
>>>
```

В последнем случае мы обратились непосредственно к ключевому полю id.

Создадим еще одно объявление:

```
>>> b2 = Bb()
>>> b2.title = 'Автомобиль'
>>> b2.content = '"Жигули"'
>>> b2.price = 1000000
>>> b2.save()
>>> > b2.pk
File "<console>", line 1
> b2.pk
^
SyntaxError: invalid syntax
>>> > b2.pk
File "<console>", line 1
> b2.pk
^
SyntaxError: invalid syntax
>>> b2.pk
2
>>>
```

Слишком мало информации, дополним:

```
>>> b2.content = '"Жигули", 1980 года, ржавая, некрашеная, сильно битая'
>>> b2.save()
```

```
>>> b2.content
'"Жигули", 1980 года, ржавая, некрашенная, сильно битая'
>>> b2.price
10000000
>>> █
```

Добавим еще одно объявление:

```
>>> Bb.objects.create(title='Дом', content='Трехэтажный, кирпич',
... price=500000000)
<Bb: Bb object (3)>
█
```

Выведем ключи и заголовки всех объявлений, имеющих в модели Bb:

```
>>> for b in Bb.objects.all():
...     print(b.pk, ': ', b.title)
...
1 :   Дача
2 :   Автомобиль
3 :   Дом
>>> █
```

Отсортируем записи модели по заголовку:

```
>>> for b in Bb.objects.order_by('title'):
...     print(b.pk, ': ', b.title)
...
2 :   Автомобиль
1 :   Дача
3 :   Дом
>>>
```

Извлечем объявления о продаже домов:

```
>>> for b in Bb.objects.filter(title='Дом'):
...     print(b.pk, ': ', b.title)
...
3 :   Дом
>>>
```

Объявление о продаже автомобиля имеет ключ 2. Отыщем его:

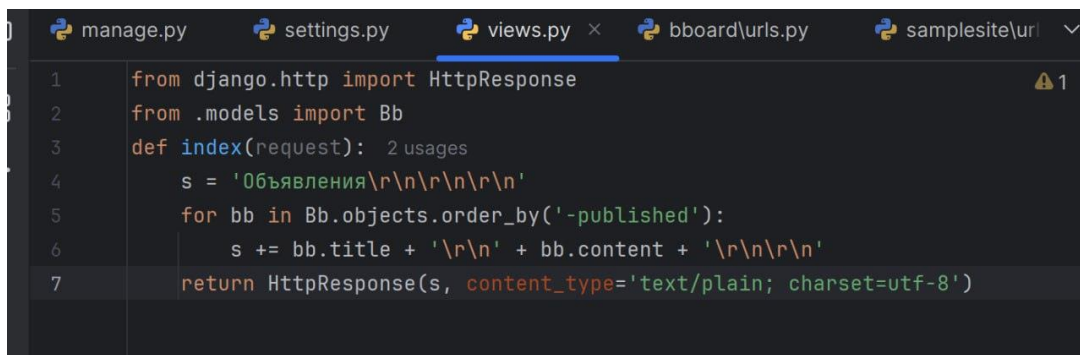
```
>>> b = Bb.objects.get(pk=2)
>>> b.title
'Автомобиль'
>>> b.content
'"Жигули", 1980 года, ржавая, некрашеная, сильно битая'
>>>
```

Метод `get()` диспетчера записей имеет то же назначение, что и метод `filter()`, и вызывается аналогичным образом. Однако он ищет не все подходящие записи, а лишь одну и возвращает ее в качестве результата.

Удалили это позорище:

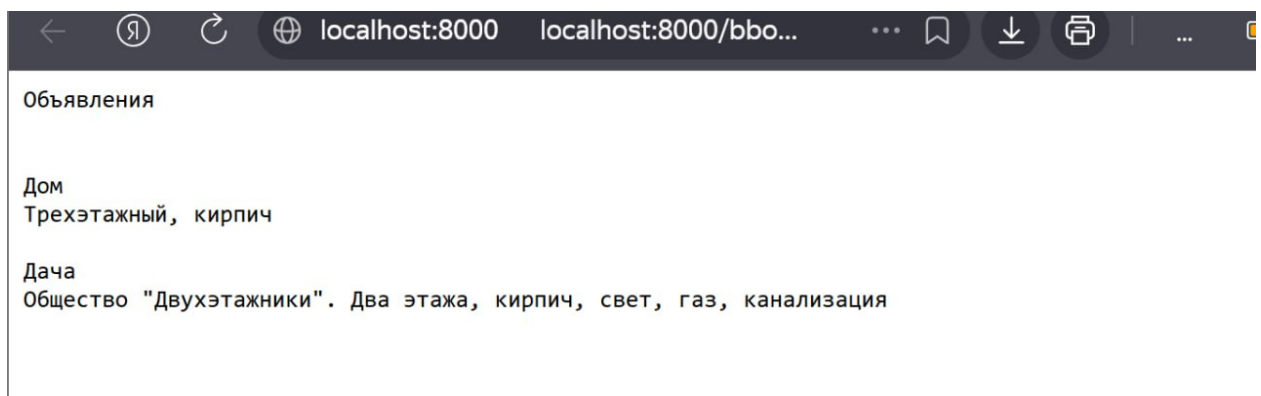
```
>>> b.delete()
(1, {'bboard.Bb': 1})
```

Откроем модуль `views.py` пакета приложения `bboard` и исправим хранящийся в нем код согласно листингу 1.8.



```
1 from django.http import HttpResponse
2 from .models import Bb
3 def index(request):
4     s = 'Объявления\r\n\r\n\r\n'
5     for bb in Bb.objects.order_by('-published'):
6         s += bb.title + '\r\n' + bb.content + '\r\n\r\n'
7     return HttpResponse(s, content_type='text/plain; charset=utf-8')
```

Сохранили исправленный модуль и запустили отладочный веб-сервер



Объявления

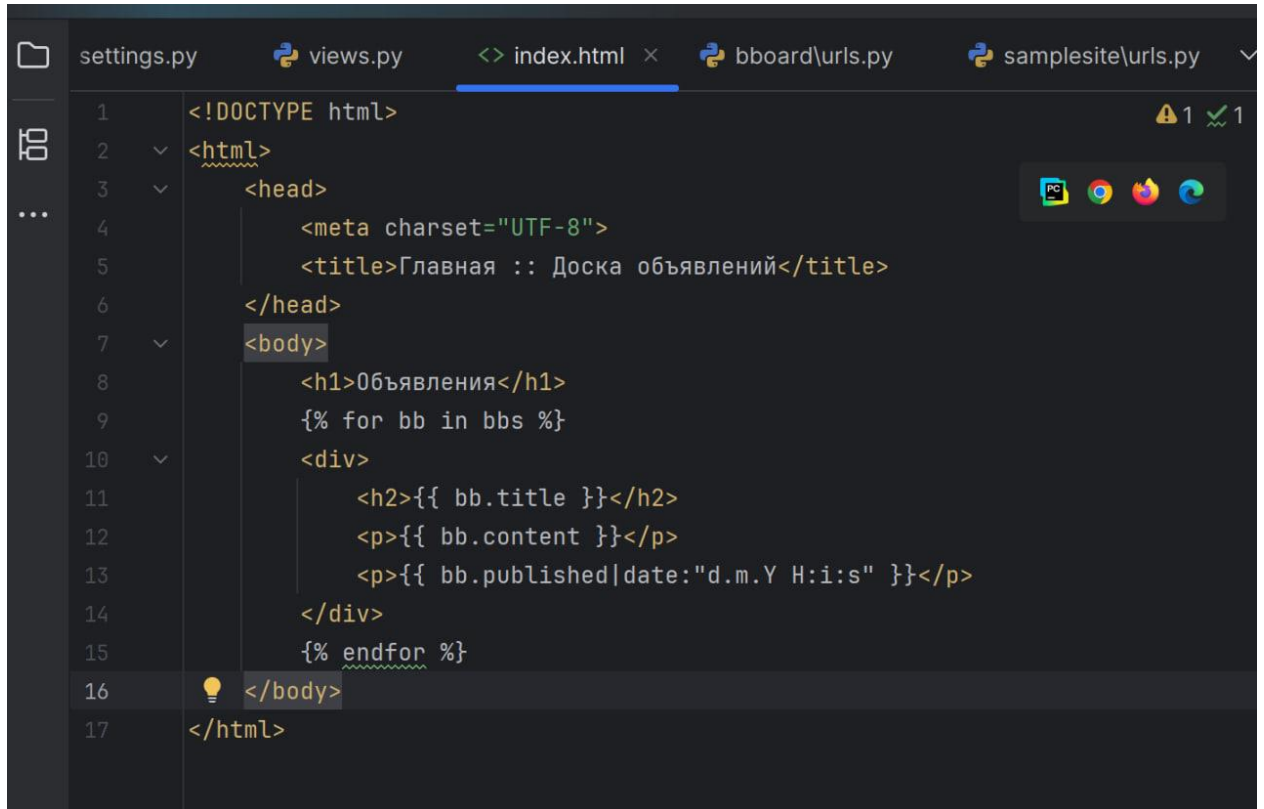
Дом  
Трехэтажный, кирпич

Дача  
Общество "Двухэтажники". Два этажа, кирпич, свет, газ, канализация

Вывод списка объявлений в виде обычного текста

## 1.11. Шаблоны

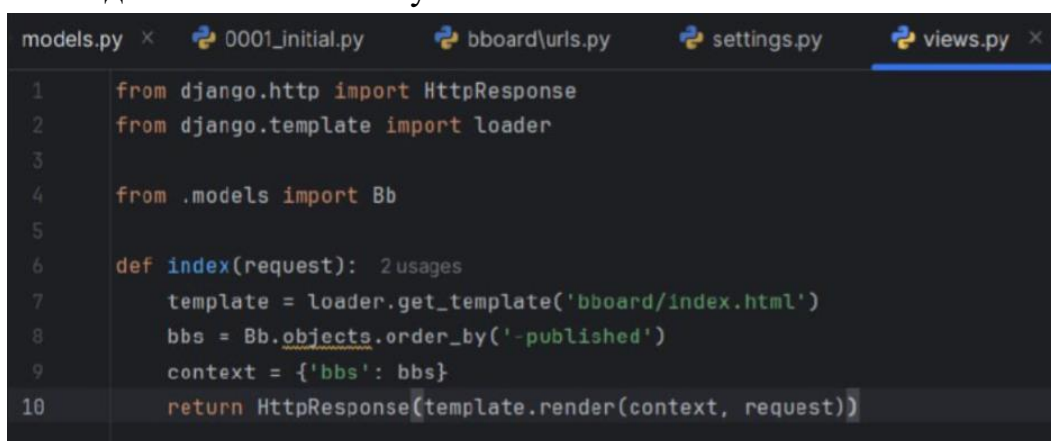
Остановили отладочный сервер. Создали в папке пакета приложения bboard папку templates, а в ней — вложенную папку bboard. Затем сохранили в этой папке наш первый шаблон index.html, код которого приведен в листинге 1.9.



```
1 <!DOCTYPE html>
2 <html>
3 <head>
4     <meta charset="UTF-8">
5     <title>Главная :: Доска объявлений</title>
6 </head>
7 <body>
8     <h1>Объявления</h1>
9     {% for bb in bbs %}
10    <div>
11        <h2>{{ bb.title }}</h2>
12        <p>{{ bb.content }}</p>
13        <p>{{ bb.published|date:"d.m.Y H:i:s" }}</p>
14    </div>
15    {% endfor %}
16 </body>
17 </html>
```

## 1.12. Контекст шаблона, рендеринг и сокращения

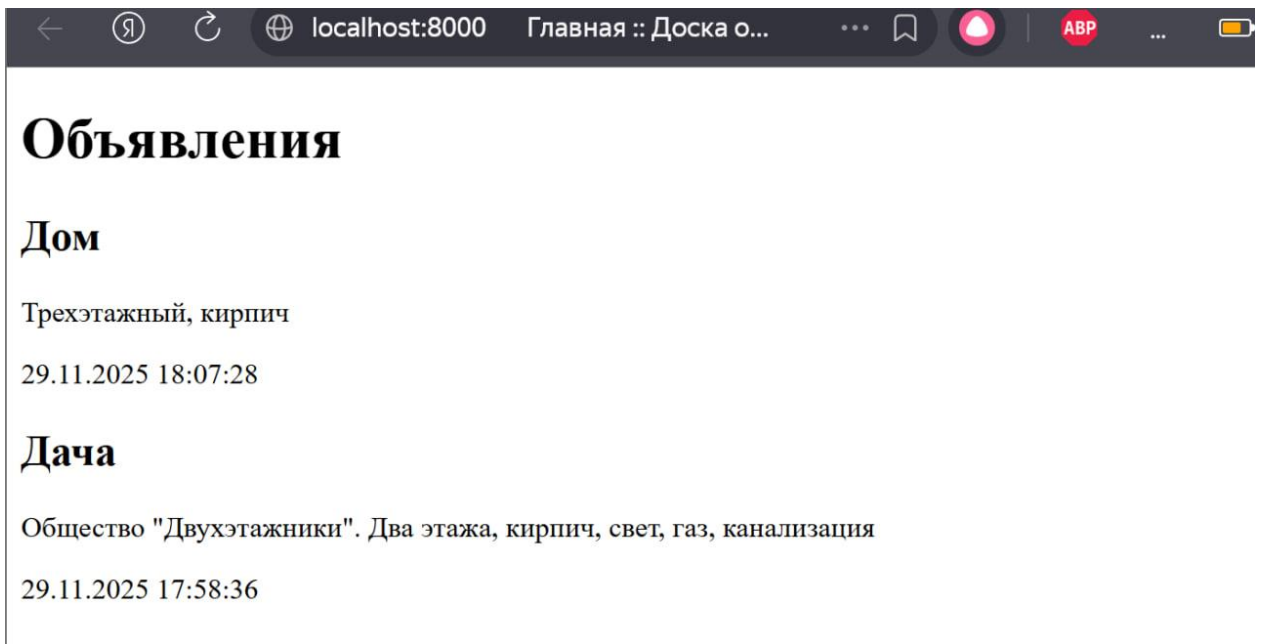
Открыли модуль views.py пакета приложения bboard и внесли исправления в его код согласно листингу 1.10



```
1 from django.http import HttpResponse
2 from django.template import loader
3
4 from .models import Bb
5
6 def index(request): 2 usages
7     template = loader.get_template('bboard/index.html')
8     bbs = Bb.objects.order_by('-published')
9     context = {'bbs': bbs}
10    return HttpResponse(template.render(context, request))
```

Сохранили оба исправленных файла, запустили отладочный веб-сервер и посмотрели на результат. Теперь это действительно веб-страница:





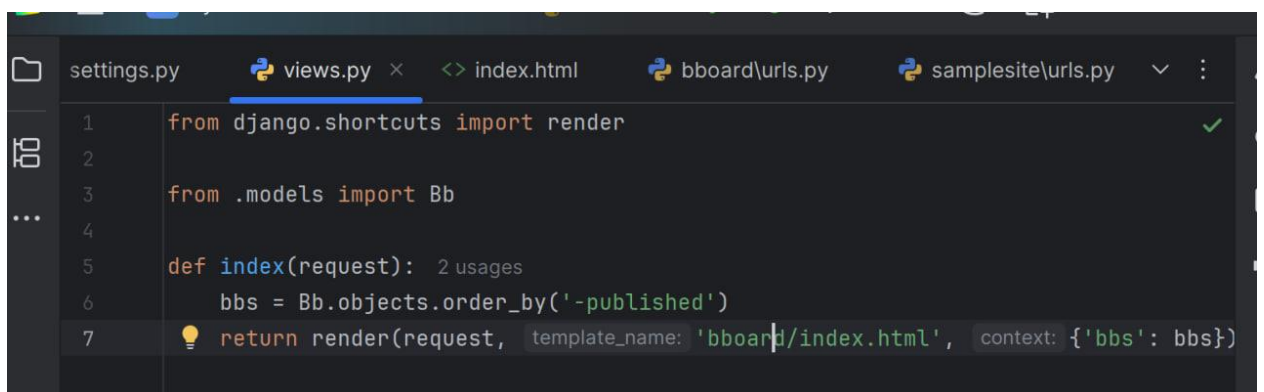
Веб-страница, сформированная с применением шаблона

В коде контроллера `index()` (см. листинг 1.10) для рендеринга мы использовали низкоуровневые инструменты, несколько усложнив код.

Но Django предоставляет средства более высокого уровня — функции-сокращения (shortcuts).

Так, функциясокращение `render()` из модуля `django.shortcuts` выполняет и загрузку, и рендеринг шаблона.

Попробовали ее в деле, исправив код модуля `views.py`, как показано в листинге 1.11.:



Обновили веб-страницу со списком объявлений и убедились, что этот код работает точно так же, как и написанный ранее, однако при этом имеет меньший объем.

## 1.13. Административный веб-сайт Django

Два объявления — это слишком мало, поэтому добавили еще несколько.

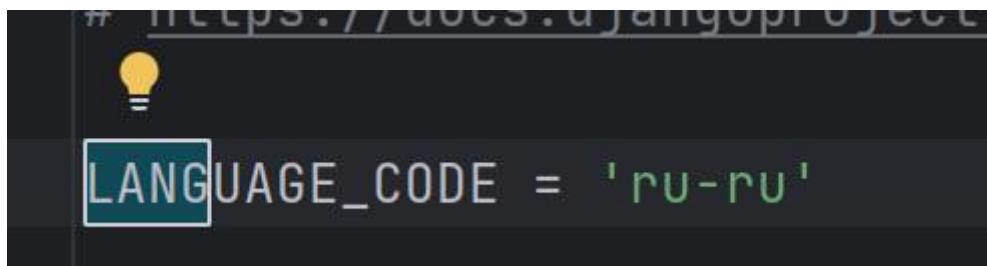
Сделали это не в консоли Django, а на встроенном в этот фреймворк административном сайте.

Создали зарегистрированного пользователя сайта с максимальными правами — суперпользователя. Для этого остановили отладочный веб-сервер и отдали в командной строке команду:

```
Username (leave blank to use 'za1ts'): admin
Email address:
Password:
Password (again):
Error: Blank passwords aren't allowed.
Password:
Password (again):
Error: Blank passwords aren't allowed.
Password:
Password (again):
Error: Your passwords didn't match.
Password:
Password (again):
This password is too short. It must contain at least 8 characters.
This password is too common.
This password is entirely numeric.
Bypass password validation and create user anyway? [y/N]: y
Superuser created successfully.
(.venv) PS D:\PycharmProjects\PythonProject\samplesite>
```

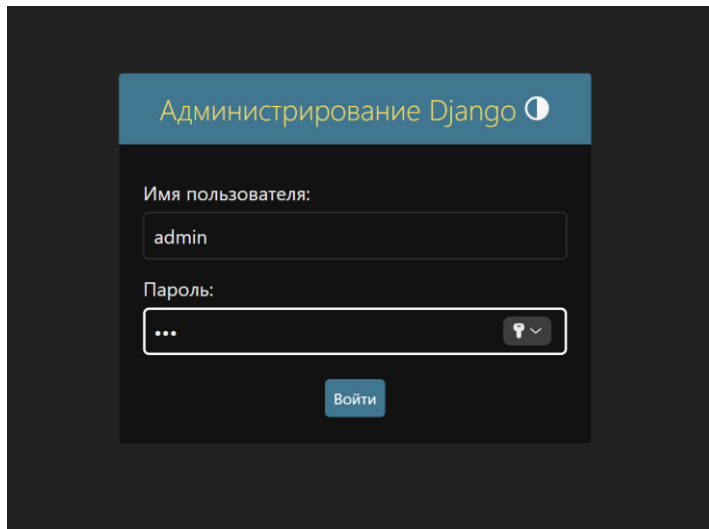
После создания суперпользователя русифицировали проект Django.

Открыли модуль settings.py пакета конфигурации и нашли в нем вот такое выражение:



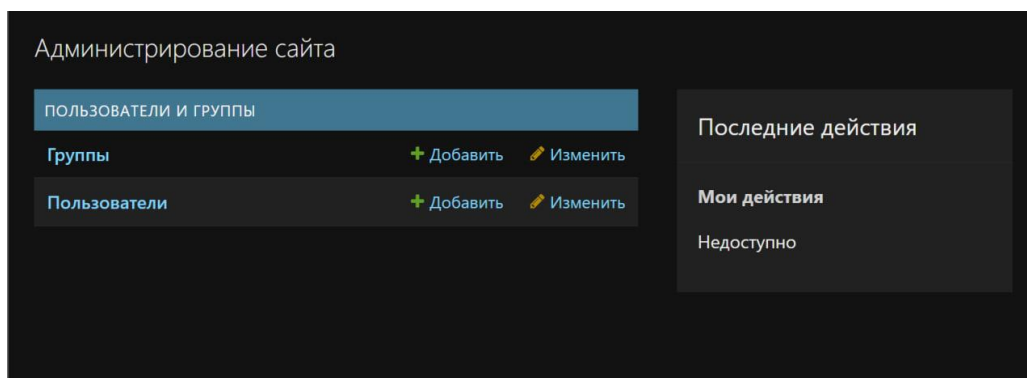
The image shows a snippet of Python code from a file named settings.py. A yellow lightbulb icon is positioned above the line of code. The code is: `LANGUAGE_CODE = 'ru-ru'`. The text 'ru-ru' is highlighted in green. The background is dark, and the text is light gray.

После успешного входа вывели страницу со списком приложений, зарегистрированных в проекте, и объявленных в этих приложениях моделей (рис. 1.6)

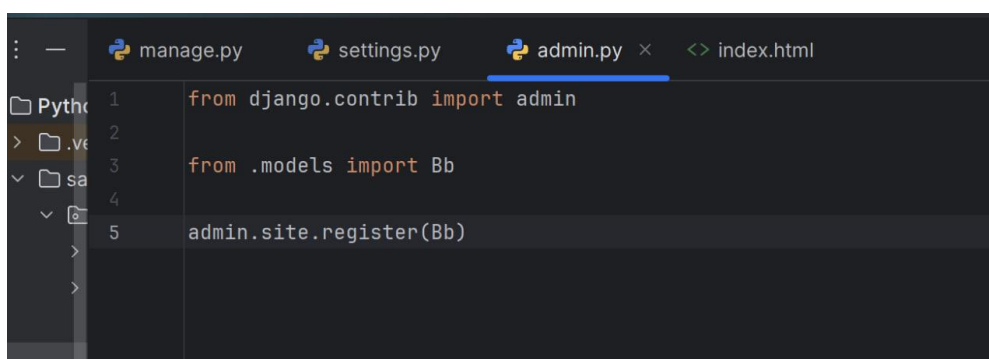


Веб-страница входа административного веб-сайта Django

Веб-страница списка приложений административного веб-сайта:

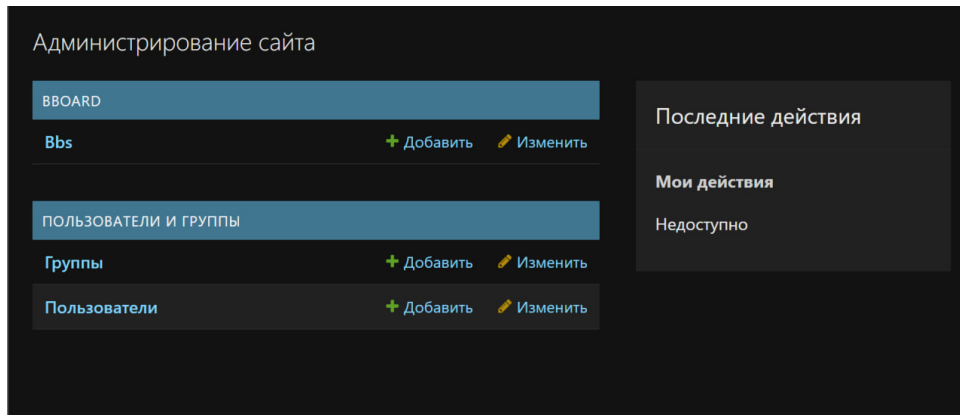


Чтобы приложение появилось в списке административного сайта, его нужно явно зарегистрировать там. Для этого открыли модуль административных настроек `admin.py` пакета приложения `bboard` и заменили его содержимое кодом, представленным в листинге 1.12.

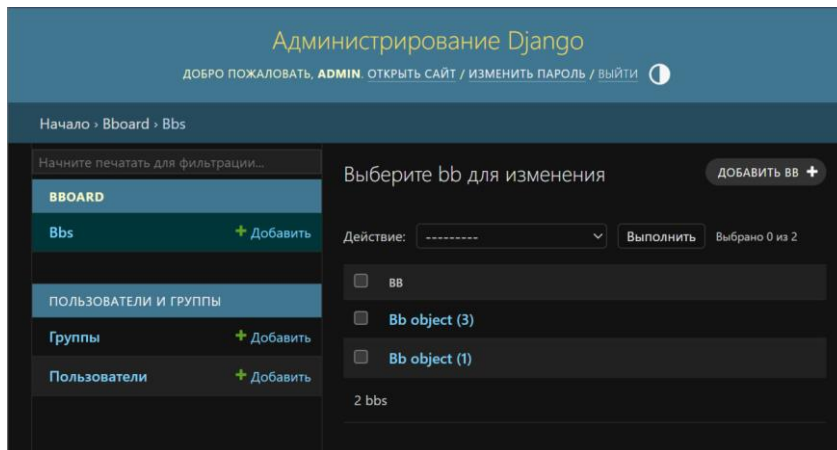




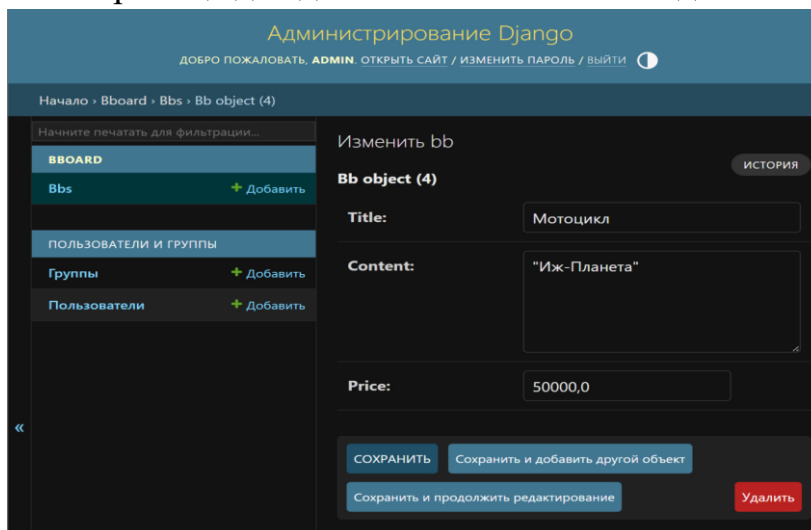
Как только мы сохранили модуль и обновили открытую в веб-обозревателе страницу списка приложений, сразу увидели, что наше приложение также присутствует в списке (рис. 1.7).



Приложение bboard в списке приложений административного веб-сайта. Список записей, хранящихся в модели Bb:

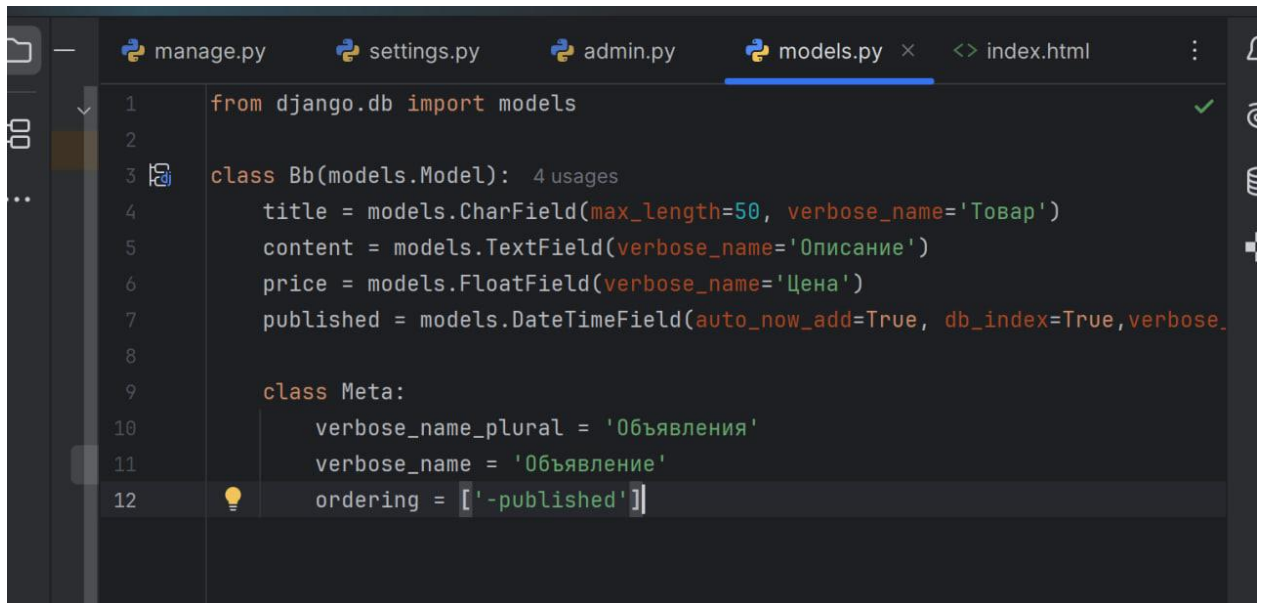


Веб-страница для добавления записи в модель:



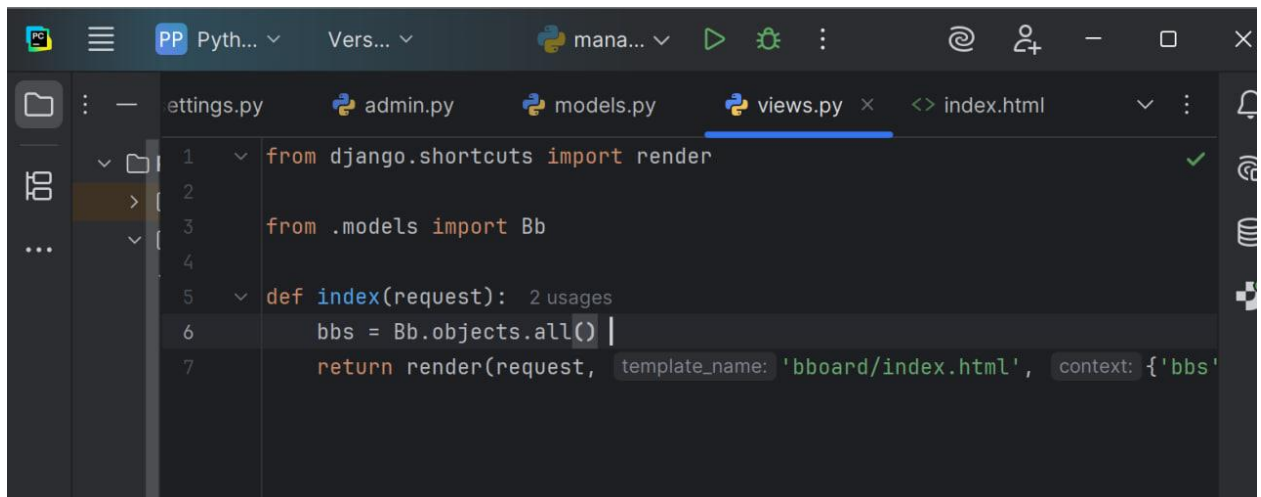
## 1.14. Параметры полей и моделей

Нужно задать параметры как для полей модели, так и для самой модели. Для этого открыли модуль `models.py` пакета приложения `bboard` и исправили код класса модели `Bb`, как показано в листинге 1.13.



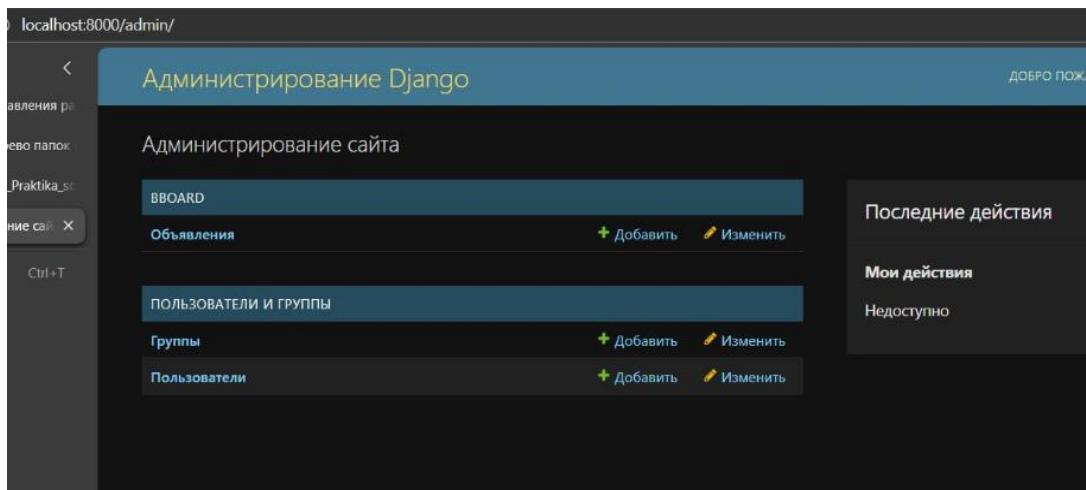
```
1 from django.db import models
2
3 class Bb(models.Model):
4     title = models.CharField(max_length=50, verbose_name='Товар')
5     content = models.TextField(verbose_name='Описание')
6     price = models.FloatField(verbose_name='Цена')
7     published = models.DateTimeField(auto_now_add=True, db_index=True, verbose_name='Дата публикации')
8
9     class Meta:
10         verbose_name_plural = 'Объявления'
11         verbose_name = 'Объявление'
12         ordering = ['-published']
```

Кстати, теперь мы можем немного упростить код контроллера `index()` (объявленного в модуле `views.py` пакета приложения), убрав из выражения, извлекающего список записей, указание сортировки:



```
1 from django.shortcuts import render
2
3 from .models import Bb
4
5 def index(request):
6     bbs = Bb.objects.all()
7     return render(request, 'bboard/index.html', context={'bbs': bbs})
```

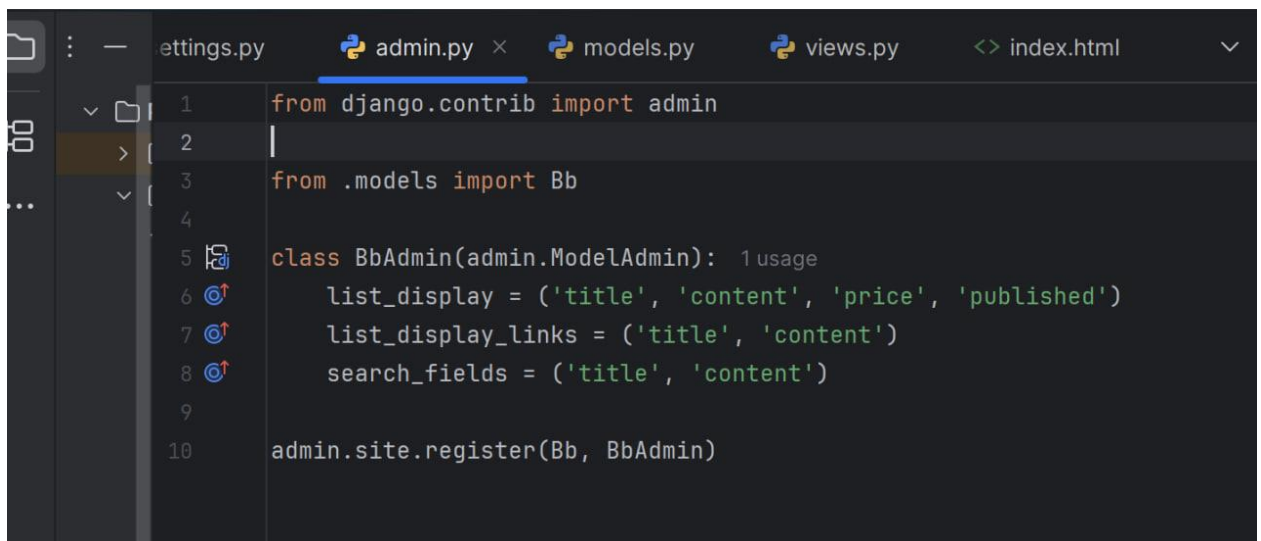
Мы сохранили исправленный модуль и обновили открытую в веб-обозревателе страницу и увидели, что:



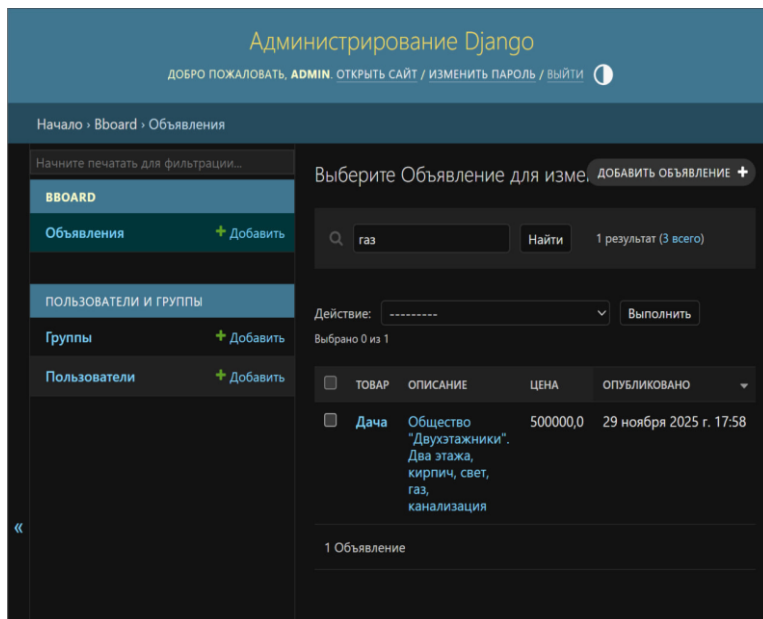
## 1.15. Редактор модели

Редактор объявляется в модуле административных настроек `admin.py` пакета приложения.

Открыли его и заменили имеющийся в нем код фрагментом, показанным в листинге 1.14.



Список записей модели Bb после указания для нее редактора (была выполнена фильтрация по слову «газ»)



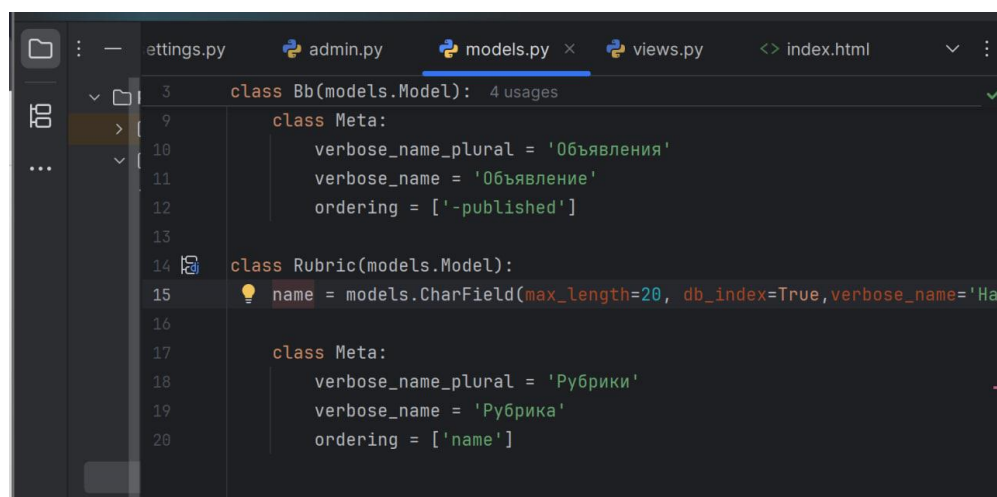
## Глава 2

### Связи. Ввод данных. Статические файлы

#### 2.1. Связи между моделями

Создадим тематические рубрики.

Сначала объявили класс модели `Rubric`, которая будет представлять рубрики объявлений, дописав в модуль `models.py` пакета приложения `bboard` код из листинга 2.1.



После создания связи «один-со-многими», при которой одна запись модели `Rubric` (рубрика) будет связана с произвольным количеством записей модели `Bb` (объявлений). Модель `Rubric` станет первичной, а `Bb` — вторичной.

Создали во вторичной модели такое поле, назвав его rubric:

```
class Bb(models.Model):
    title = models.CharField(max_length=50, verbose_name='Товар')
    content = models.TextField(verbose_name='Описание')
    price = models.FloatField(verbose_name='Цена')
    published = models.DateTimeField(auto_now_add=True, db_index=True, verbose_name='Опубликовано')
    rubric = models.ForeignKey(to='Rubric', null=True,
                              on_delete=models.PROTECT, verbose_name='Рубрика')

    class Meta:
        verbose_name_plural = 'Объявления'
```

Сохранили исправленный модуль и сгенерировали миграции, которые внесут необходимые изменения в базу данных:

```
django.db.utils.OperationalError: no such column: bboard_bb.rubric_id
[29/Nov/2025 22:31:32] "GET /admin/bboard/bb/?q=%D0%B3%D0%B0%D0%B7 HTTP/1.1" 500 172635
(.venv) PS D:\PycharmProjects\PythonProject\samplesite> python manage.py makemigrations bboard
Migrations for 'bboard':
  bboard\migrations\0002_rubric_alter_bb_options_alter_bb_content_and_more.py
  + Create model Rubric
  ~ Change Meta options on bb
  ~ Alter field content on bb
  ~ Alter field price on bb
  ~ Alter field published on bb
  ~ Alter field title on bb
  + Add field rubric to bb
(.venv) PS D:\PycharmProjects\PythonProject\samplesite>
```

Новая миграция создаст таблицу для модели Rubric и добавит в таблицу модели Bb новое поле rubric.

Выполнили созданную миграцию:

```
+ Add field rubric to bb
(.venv) PS D:\PycharmProjects\PythonProject\samplesite> python manage.py migrate
Operations to perform:
  Apply all migrations: admin, auth, bboard, contenttypes, sessions
Running migrations:
  Applying bboard.0002_rubric_alter_bb_options_alter_bb_content_and_more... OK
(.venv) PS D:\PycharmProjects\PythonProject\samplesite>
```

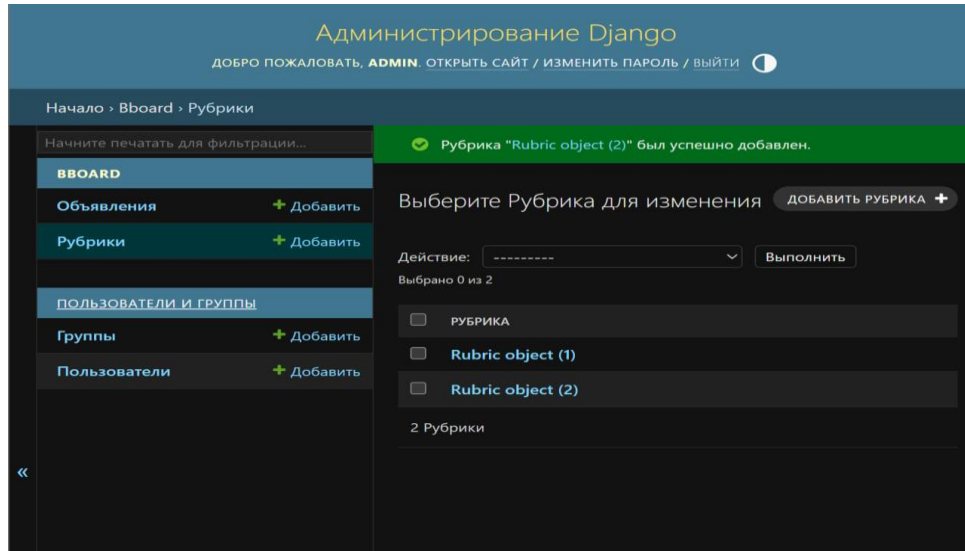
Сразу же зарегистрировали новую модель на административном сайте, внося в модуль admin.py пакета приложения следующие правки:

```
from django.contrib import admin
from .models import Bb, Rubric

class BbAdmin(admin.ModelAdmin):
    list_display = ('title', 'content', 'price', 'published')
    list_display_links = ('title', 'content')
    search_fields = ('title', 'content')

admin.site.register(Bb, BbAdmin)
admin.site.register(Rubric)
```

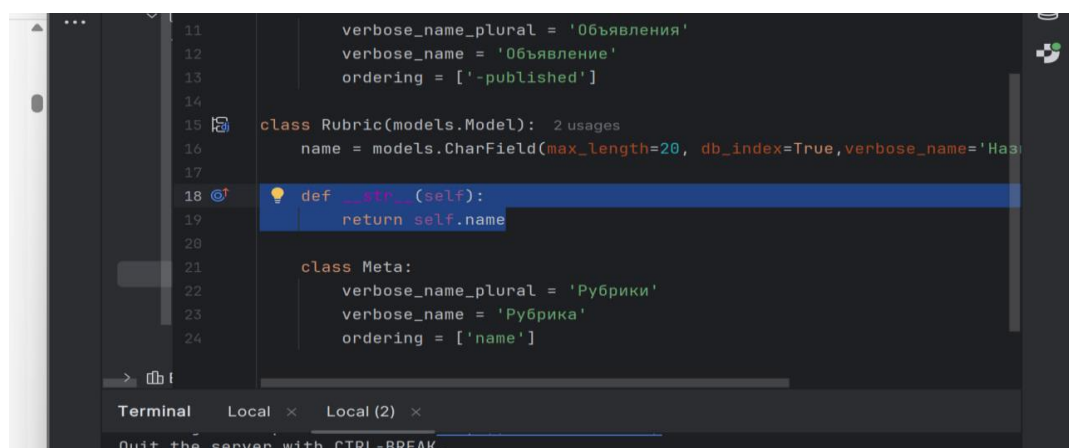
Запустили отладочный веб-сервер, вошли на административный сайт и добавили в модель Rubric рубрики «Недвижимость» и «Транспорт»:



## 2.2. Строковое представление модели

Можно переопределить в классе модели метод `__str__(self)`, возвращающий строковое представление класса, а именно название рубрики. Так и сделали.

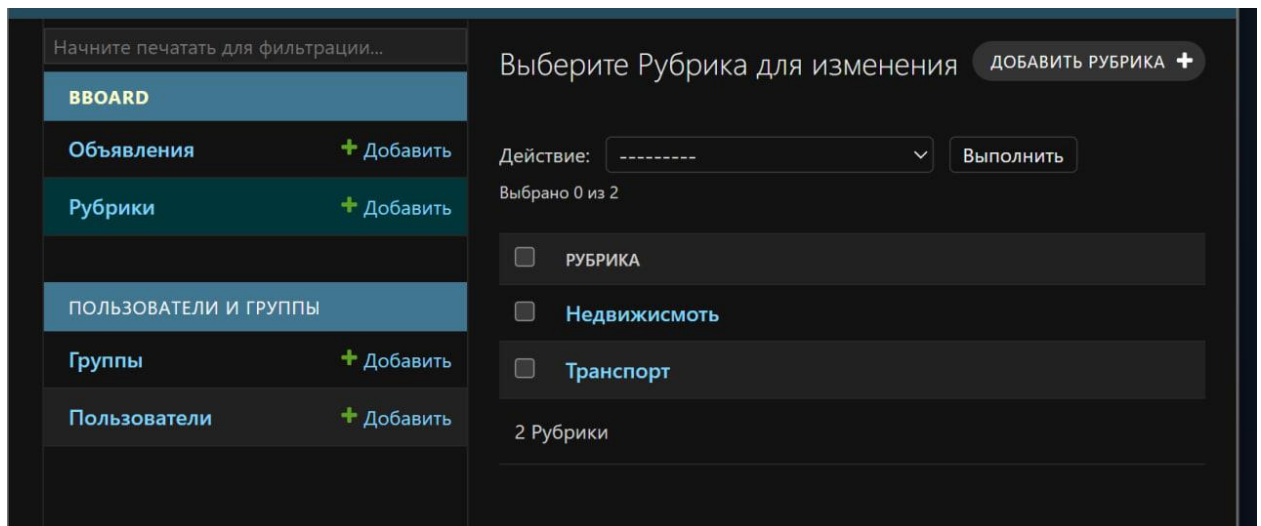
Открыли модуль `models.py` и добавили в объявление класса модели `Rubric` ВОТ ЭТОТ КОД:



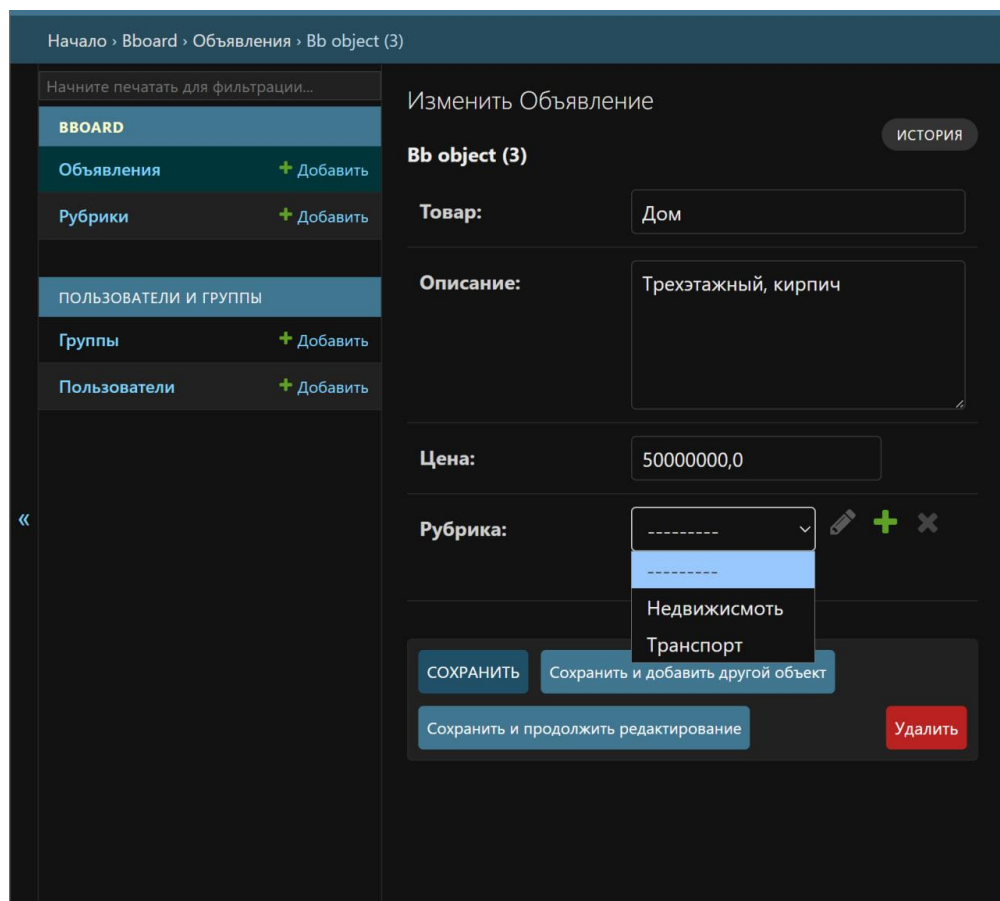
Сохранили модуль, обновили страницу списка рубрик в административном сайте и посмотрели, что у нас получилось.

Список рубрик (выводятся строковые представления записей модели)





Для указания значения поля внешнего ключа применяется раскрывающийся список:



Осталось сделать так, чтобы в списке записей модели Bb, помимо всего прочего, выводились рубрики объявлений, и пусть они выводятся в самом первом столбце.

Для чего вставили в начало последовательности имен полей, присвоенной атрибуту `list_display` класса `BbAdmin`, поле `rubric`:

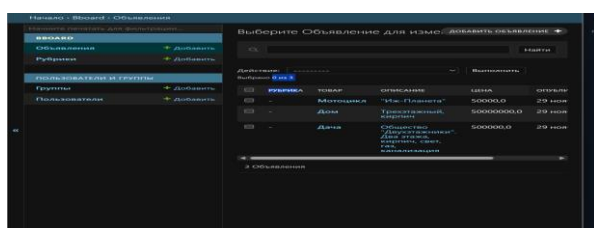


```

3 from .models import Bb, Rubric
4
5 class BbAdmin(admin.ModelAdmin):
6     list_display = ('rubric', 'title', 'content', 'price', 'published')
7     list_display_links = ('title', 'content')
8     search_fields = ('title', 'content')
9
10 admin.site.register(Bb, BbAdmin)
11 admin.site.register(Rubric)
12

```

Обновили страницу списка объявления — и сразу увидели в нем новый столбец Рубрика.



### 2.3. URL-параметры и параметризованные запросы

Достаточно указать маршрутизатору, какую часть интернет-адреса считать URL-параметром, каков тип значения этого параметра и какое имя должно быть у параметра контроллера, которому будет присвоено значение URL-параметра, извлеченного из адреса.

Открыли модуль `urls.py` пакета приложения `bboard` и внесли в него такие правки:

```

admin.py  bboard\urls.py  samplesite\urls.py  views.py
1 from django.urls import path
2 from .views import index, rubric_bbs
3 urlpatterns = [
4     path(route: '<int:rubric_id>/', rubric_bbs),
5     path(route: '', index),
6 ]

```

Открыли модуль `views.py` и добавили в него код контроллера-функции `rubric_bbs()` (листинг 2.2).

```

admin.py  urls.py  views.py  models.py  index.html
1  from django.shortcuts import render
2
3  from .models import Bb
4  from .models import Rubric
5
6  def index(request):
7      bbs = Bb.objects.all()
8      return render(request, 'bboard/index.html', context={'bbs': bbs})
9
10 def rubric_bbs(request, rubric_id):
11     bbs = Bb.objects.filter(rubric=rubric_id)
12     rubrics = Rubric.objects.all()
13     current_rubric = Rubric.objects.get(pk=rubric_id)
14     context = {'bbs': bbs, 'rubrics': rubrics, 'current_rubric': current_rubric}
15     return render(request, 'bboard/rubric_bbs.html', context)
16

```

Создали в папке `templates\bboard` пакета приложения шаблон `rubric_bbs.html` с кодом, приведенным в листинге 2.3.

```

admin.py  urls.py  views.py  rubric_bbs.html  models.py  index.html
1  <!DOCTYPE html>
2  <html>
3      <head>
4          <meta charset="UTF-8">
5          <title>{{ current_rubric.name }} :: Доска объявлений</title>
6      </head>
7      <body>
8          <h1>Объявления</h1>
9          <h2>Рубрика: {{ current_rubric.name }}</h2>
10         <div>
11             <a href="/bboard/">Главная</a>
12             {% for rubric in rubrics %}
13             <a href="/bboard/{{ rubric.pk }}/">{{ rubric.name }}</a>
14             {% endfor %}
15         </div>
16         {% for bb in bbs %}
17         <div>
18             <h2>{{ bb.title }}</h2>
19             <p>{{ bb.content }}</p>
20             <p>{{ bb.published|date:"d.m.Y H:i:s" }}</p>
21         </div>
22         {% endfor %}
23     </body>
24 </html>

```

Исправили

контроллер `index()` и шаблон `bboard/index.html` таким образом, чтобы на главной странице нашего сайта выводилась та же самая панель навигации.

Помимо этого, сделали так, чтобы в составе каждого объявления выводилось название рубрики, к которой оно относится, выполненное в виде гиперссылки.

Код исправленного контроллера `index()` показан в листинге 2.4.

```

admin.py  urls.py  views.py  rubric_bbs.html  index.html  models.py
1  from django.shortcuts import render
2
3  from .models import Bb
4  from .models import Rubric
5
6  def index(request):
7      bbs = Bb.objects.all()
8      rubrics = Rubric.objects.all()
9      context = {'bbs': bbs, 'rubrics': rubrics}
10     return render(request, 'bboard/index.html', context)
11
12 def rubric_bbs(request, rubric_id):
13     bbs = Bb.objects.filter(rubric=rubric_id)
14     rubrics = Rubric.objects.all()
15     current_rubric = Rubric.objects.get(pk=rubric_id)
16     context = {'bbs': bbs, 'rubrics': rubrics, 'current_rubric': current_rubric}
17     return render(request, 'bboard/rubric_bbs.html', context)
18

```

Полный код исправленного шаблона bboard/index.html приведен в листинге 2.5.

```

admin.py  urls.py  views.py  rubric_bbs.html  index.html  models.py
1  <!DOCTYPE html>
2  <html>
3  <head>
4      <meta charset="UTF-8">
5      <title>Главная :: Доска объявлений</title>
6  </head>
7  <body>
8      <h1>Объявления</h1>
9      <div>
10         <a href="/bboard/">Главная</a>
11         {% for rubric in rubrics %}
12         <a href="/bboard/{{ rubric.pk }}/">{{ rubric.name }}</a>
13         {% endfor %}
14     </div>
15     {% for bb in bbs %}
16     <div>
17         <h2>{{ bb.title }}</h2>
18         <p>{{ bb.content }}</p>
19         <p><a href="/bboard/{{ bb.rubric.pk }}/">
20             {{ bb.rubric.name }}</a></p>
21         <p>{{ bb.published|date:"d.m.Y H:i:s" }}</p>
22     </div>
23     {% endfor %}
24 </body>
25 </html>

```

Исправленная главная веб-страница с панелью навигации и обозначениями рубрик:

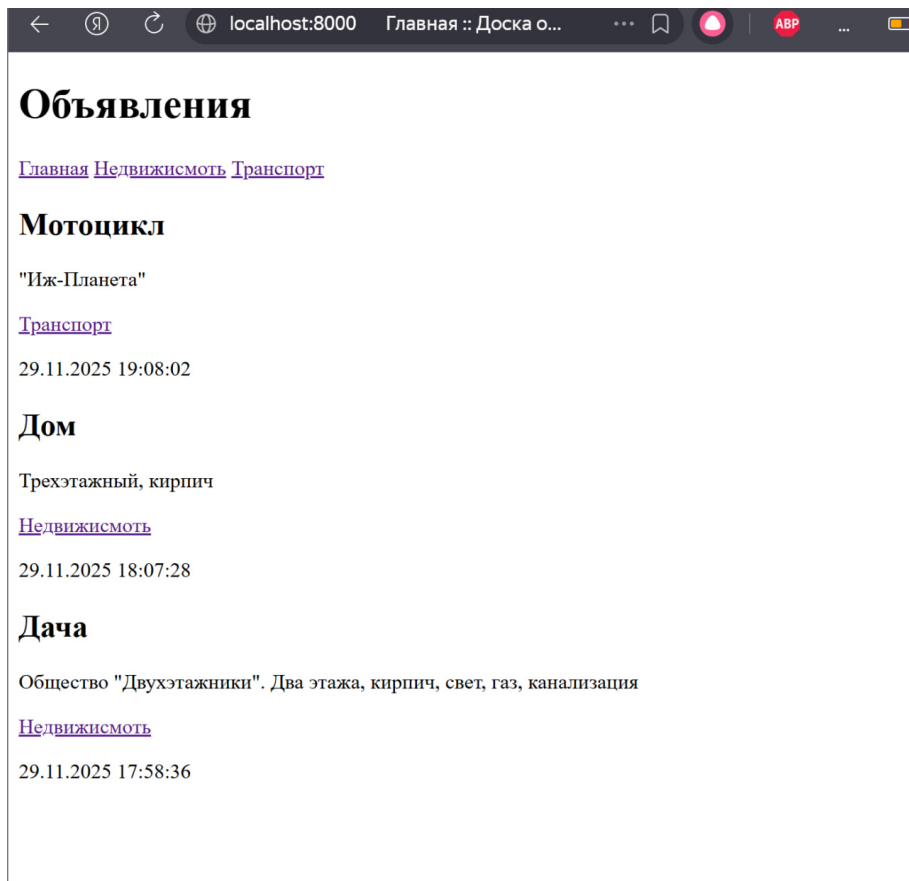


Рис 2.3.

Веб-страница объявлений, относящихся к выбранной рубрике:

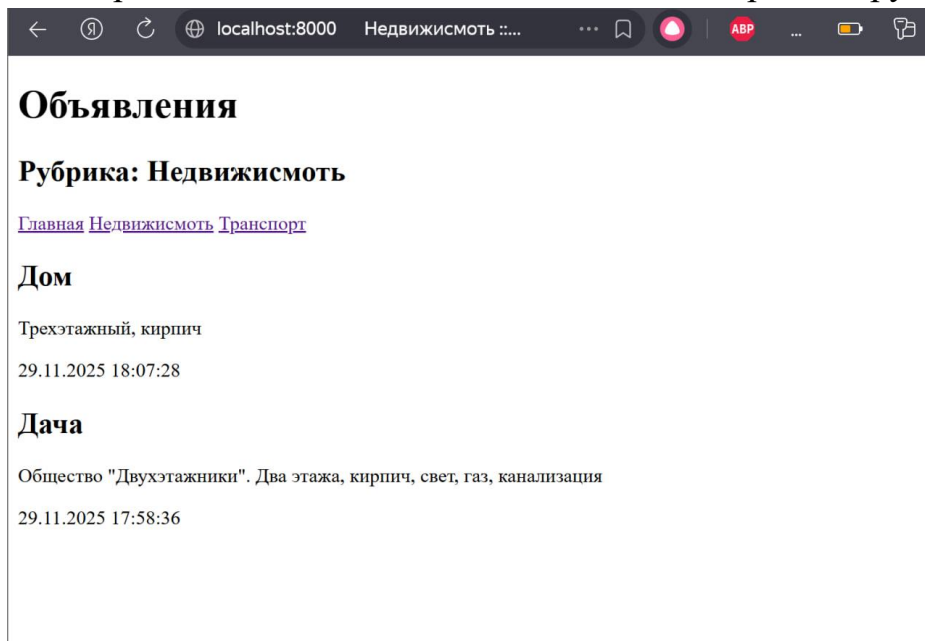


Рис 2.4.

Сохранили исправленные файлы, запустили отладочный веб-сервер и перейдем на главную страницу.

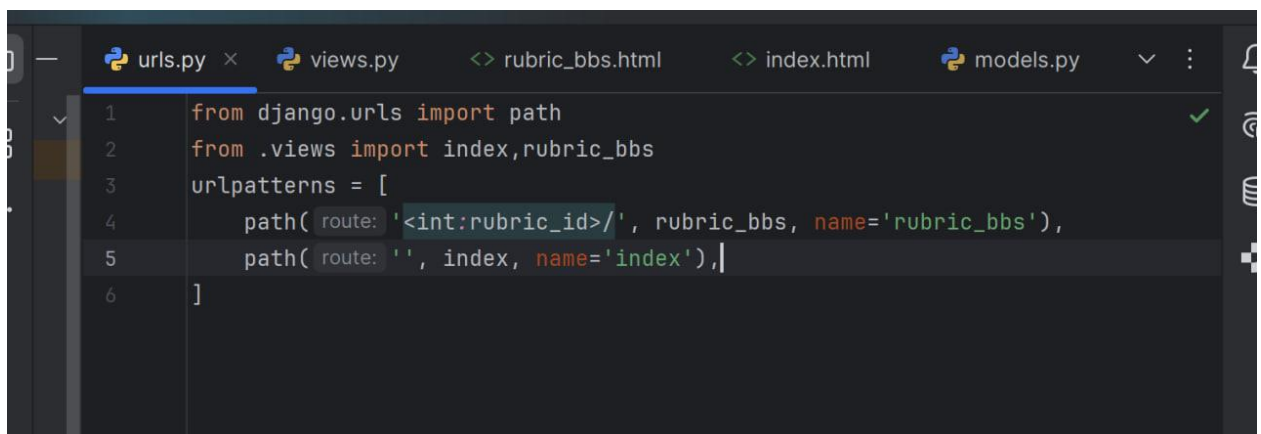
Мы сразу увидели панель навигации, располагающуюся непосредственно под заголовком, и гиперссылки на рубрики, к которым относятся объявления (рис. 2.3).

Перешли по какой-либо гиперссылке-рубрике и посмотрели на страницу со списком относящихся к ней объявлений (рис. 2.4).

## 2.4. Обратное разрешение интернет-адресов

Сначала нужно дать маршрутам имена, создав тем самым именованные маршруты.

Открыли модуль `urls.py` пакета приложения и исправили код, создающий набор маршрутов, следующим образом:



```
1 from django.urls import path
2 from .views import index, rubric_bbs
3 urlpatterns = [
4     path(route: '<int:rubric_id>/', rubric_bbs, name='rubric_bbs'),
5     path(route: '', index, name='index'),
6 ]
```

Далее следует вставить в код гиперссылок теги шаблонизатора `url`, которые и выполняют обратное разрешение интернет-адресов.

Открыли шаблон `bboard\index.html`, нашли в нем фрагмент кода:

```
admin.py  urls.py  views.py  <> rubric_bbs.html  <> index.html x  models.py

1  <!DOCTYPE html>
2  <html>
3      <head>
4          <meta charset="UTF-8">
5          <title>Главная :: Доска объявлений</title>
6      </head>
7      <body>
8          <h1>Объявления</h1>
9          <div>
10             <a href="{% url 'index' %}">Главная</a>
11             {% for rubric in rubrics %}
12             <a href="{% url 'rubric_bbs' rubric.pk %}">{{ rubric.name }}</a>
13             {% endfor %}
14         </div>
15         {% for bb in bbs %}
16         <div>
17             <h2>{{ bb.title }}</h2>
18             <p>{{ bb.content }}</p>
19             <p><a href="{% url 'rubric_bbs' bb.rubric.pk %}">
20                 {{ bb.rubric.name }}</a></p>
21             <p>{{ bb.published|date:"d.m.Y H:i:s" }}</p>
22         </div>
23         {% endfor %}
24     </body>
25 </html>
```

Маршрут index не является параметризованным, поэтому URL-параметры здесь не указываются.

Исправили код аналогичных гиперссылок в шаблоне bboard\rubric\_bbs.html.

Обновили страницу, открытую в веб-обозревателе, и попробовали выполнить несколько переходов по гиперссылкам. Все должно работать.

```
admin.py  urls.py  views.py  rubric_bbs.html  index.html
1  <!DOCTYPE html>
2  <html>
3      <head>
4          <meta charset="UTF-8">
5          <title>{{ current_rubric.name }} :: Доска объявлений</title>
6      </head>
7      <body>
8          <h1>Объявления</h1>
9          <h2>Рубрика: {{ current_rubric.name }}</h2>
10         <div>
11             <a href="{% url 'index' %}">Главная</a>
12             {% for rubric in rubrics %}
13             <a href="{% url 'rubric_bbs' rubric.pk %}">{{ rubric.name }}</a>
14             {% endfor %}
15         </div>
16         {% for bb in bbs %}
17         <div>
18             <h2>{{ bb.title }}</h2>
19             <p>{{ bb.content }}</p>
20             <p>{{ bb.published|date:"d.m.Y H:i:s" }}</p>
21         </div>
22         {% endfor %}
23     </body>
24 </html>
```

## 2.5. Формы, связанные с моделями

Создали форму, связанную с моделью Bb и служащую для ввода новых объявлений.

Дали ее классу имя BbForm. Остановили отладочный веб-сервер. Создали в пакете приложения bboard модуль forms.py, в который занесли код из листинга 2.6.

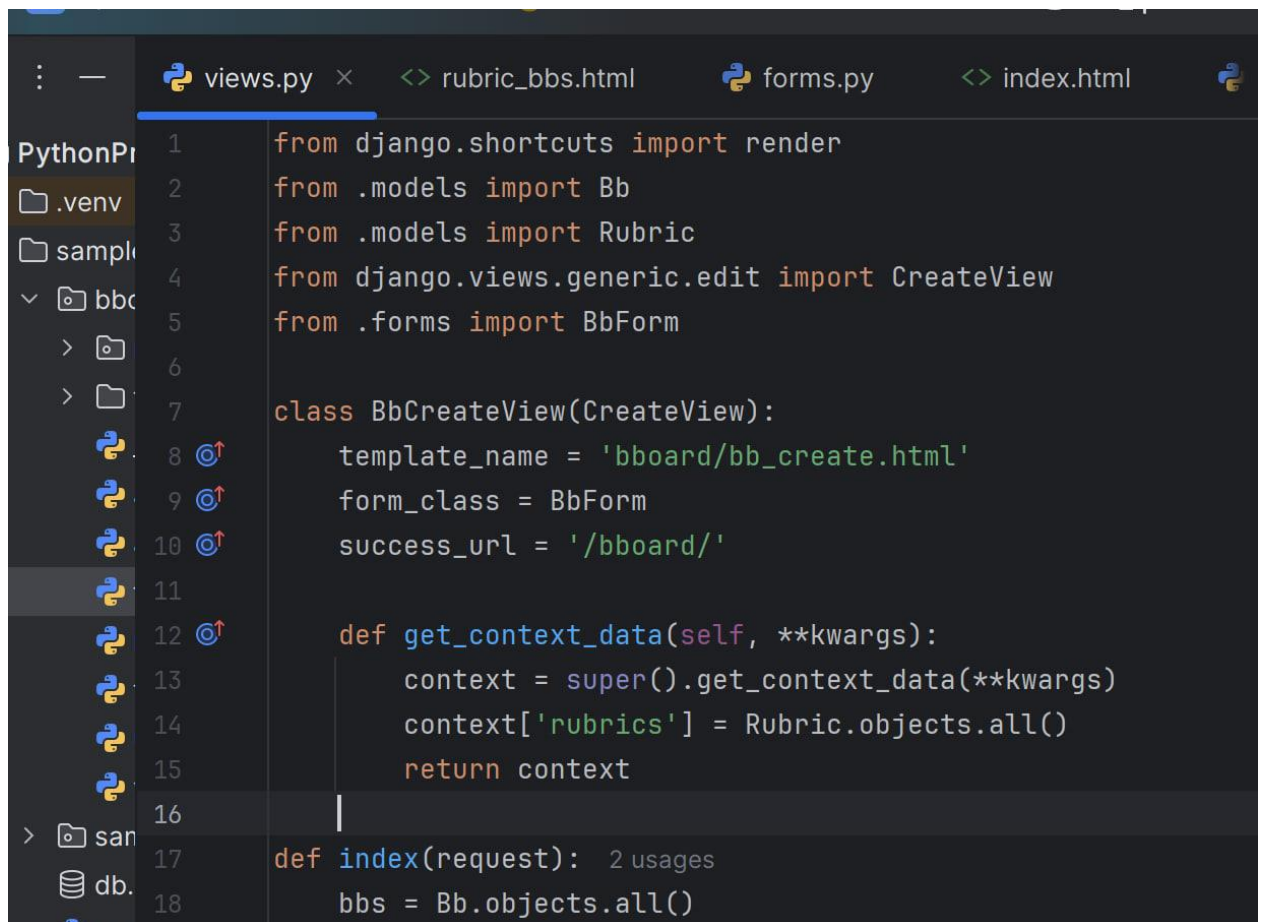
```
views.py  rubric_bbs.html  forms.py  index.html  mc
1  from django.forms import ModelForm
2
3  from .models import Bb
4
5  class BbForm(ModelForm):
6      class Meta:
7          model = Bb
8          fields = ('title', 'content', 'price', 'rubric')
```



## 2.6. Контроллеры-классы

Обрабатывать формы, связанные с моделью, можно в контроллерах-функциях. Но лучше использовать высокоуровневый контроллер-класс, который сам выполнит большую часть действий по выводу и обработке формы.

Наш первый контроллер-класс носит имя `BbCreateView`. Его мы объявили в модуле `views.py` пакета приложения. Код этого класса показан в листинге 2.7.



```
1 from django.shortcuts import render
2 from .models import Bb
3 from .models import Rubric
4 from django.views.generic.edit import CreateView
5 from .forms import BbForm
6
7 class BbCreateView(CreateView):
8     template_name = 'bboard/bb_create.html'
9     form_class = BbForm
10    success_url = '/bboard/'
11
12    def get_context_data(self, **kwargs):
13        context = super().get_context_data(**kwargs)
14        context['rubrics'] = Rubric.objects.all()
15        return context
16
17    def index(request):
18        bbs = Bb.objects.all()
```

Занялись шаблоном `bboard\bb_create.html`. Его код представлен в листинге 2,8

```
admin.py  urls.py  views.py  <> bb_create.html  <> rubric_bbs.html  forms.py  <> i
1  <!DOCTYPE html>
2  <html>
3      <head>
4          <meta charset="UTF-8">
5          <title>Добавление объявления :: Доска объявлений</title>
6      </head>
7      <body>
8          <h1>Добавление объявления</h1>
9          <div>
10             <a href="{% url 'index' %}">Главная</a>
11             {% for rubric in rubrics %}
12             <a href="{% url 'rubric_bbs' rubric.pk %}">{{ rubric.name }}</a>
13             {% endfor %}
14          </div>
15          <form method="post">
16              {% csrf_token %}
17              {{ form.as_p }}
18              <input type="submit" value="Добавить">
19          </form>
20      </body>
21  </html>
```

Здесь нас интересует только код, создающий веб-форму.

Добавили в модуль `urls.py` пакета приложения маршрут, указывающий на контроллер `CreateView`:

```
admin.py  urls.py  views.py  <> bb_create.html  <> rubric_bbs.html  fo
1  from django.urls import path
2  from .views import index, rubric_bbs
3  from .views import index, by_rubric, BbCreateView
4  urlpatterns = [
5      path(route: 'add/', BbCreateView.as_view(), name='add'),
6      path(route: '<int:rubric_id>/', rubric_bbs, name='rubric_bbs'),
7      path(route: '', index, name='index'),
8  ]
```

Не забудем создать в панели навигации всех трех страниц гиперссылку на страницу добавления объявления:

```
1 <!DOCTYPE html>
2 <html>
3   <head>
4     <meta charset="UTF-8">
5     <title>Главная :: Доска объявлений</title>
6   </head>
7   <body>
8     <h1>Объявления</h1>
9     <div>
10      <a href="{% url 'index' %}">Главная</a>
11      <a href="{% url 'add' %}">Добавить объявление</a>
12      {% for rubric in rubrics %}
13      <a href="{% url 'rubric_bbs' rubric.pk %}">{{ rubric.name }}</a>
14      {% endfor %}
15    </div>
16    {% for bb in bbs %}
17    <div>
18      <h2>{{ bb.title }}</h2>
19      <p>{{ bb.content }}</p>
20      <p><a href="{% url 'rubric_bbs' bb.rubric.pk %}">
21        {{ bb.rubric.name }}</a></p>
22      <p>{{ bb.published|date:"d.m.Y H:i:s" }}</p>
23    </div>
24    {% endfor %}
25  </body>
26 </html>
```

Веб-страница добавления нового объявления.

localhost:8000/bboard/add/

## Добавление объявления

[Главная](#) [Добавить](#) [Недвижимость](#) [Транспорт](#)

Товар:

Описание:

Цена:

Рубрика:

Открыли модуль views.py и внесли следующие правки в код класса BbCreateView:

```
from .forms import BbForm

class BbCreateView(CreateView):
    template_name = 'bboard/bb_create.html'
    form_class = BbForm
    success_url = '/bboard/'

    def get_context_data(self, **kwargs):
        context = super().get_context_data(**kwargs)
        success_url = reverse_lazy('index')
        context['rubrics'] = Rubric.objects.all()
        return context

    def index(request):
        bbs = Bb.objects.all()
        rubrics = Rubric.objects.all()
        context = {'bbs': bbs, 'rubrics': rubrics}
        return render(request, 'bboard/index.html', context)

    def rubric_bbs(request, rubric_id):
        bbs = Bb.objects.filter(rubric=rubric_id)
        rubrics = Rubric.objects.all()
        current_rubric = Rubric.objects.get(pk=rubric_id)
```

Функция `reverse_lazy()` из модуля `django.urls` принимает имя маршрута и значения всех входящих в маршрут URL-параметров (если они там есть). Результатом станет готовый интернет-адрес.

## 2.7. Наследование шаблонов

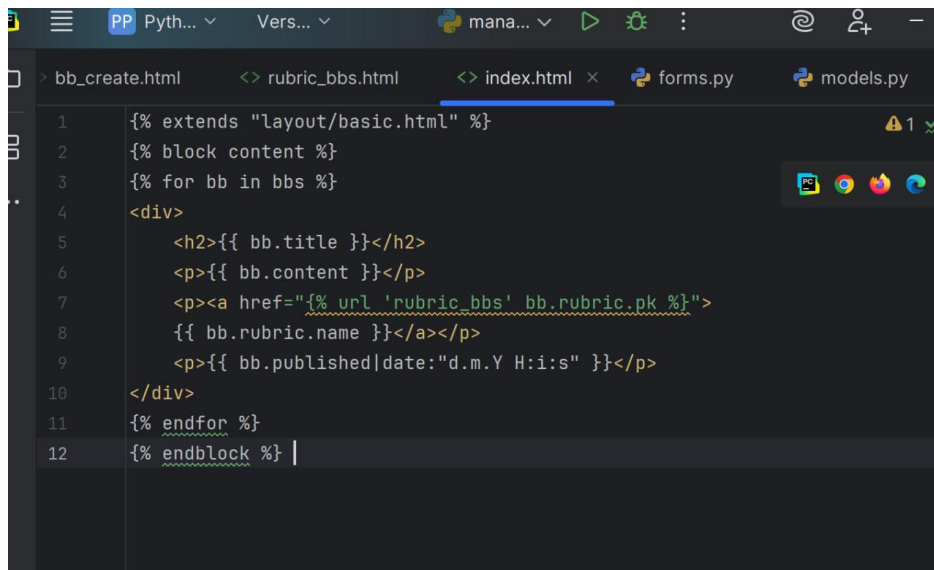
Создали в папке `templates` папку `layout`.

Сохранили в ней базовый шаблон `basic.html`, код которого приведен в листинге 2.9.

```
admin.py  urls.py  views.py  basic.html  bb_create.html  rubric_bbs.h

1  <!DOCTYPE html>
2  <html>
3      <head>
4          <meta charset="UTF-8">
5          <title>{% block title %}Главная{% endblock %} ::
6              Доска объявлений</title>
7      </head>
8      <body>
9          <header>
10             <h1>Объявления</h1>
11          </header>
12          <nav>
13              <a href="{% url 'index' %}">Главная</a>
14              <a href="{% url 'add' %}">Добавить</a>
15              {% for rubric in rubrics %}
16              <a href="{% url 'rubric_bbs' rubric.pk %}">{{ rubric.name }}</a>
17              {% endfor %}
18          </nav>
19          <main>
20              {% block content %}
21              {% endblock %}
22          </main>
23      </body>
24  </html>
25
```

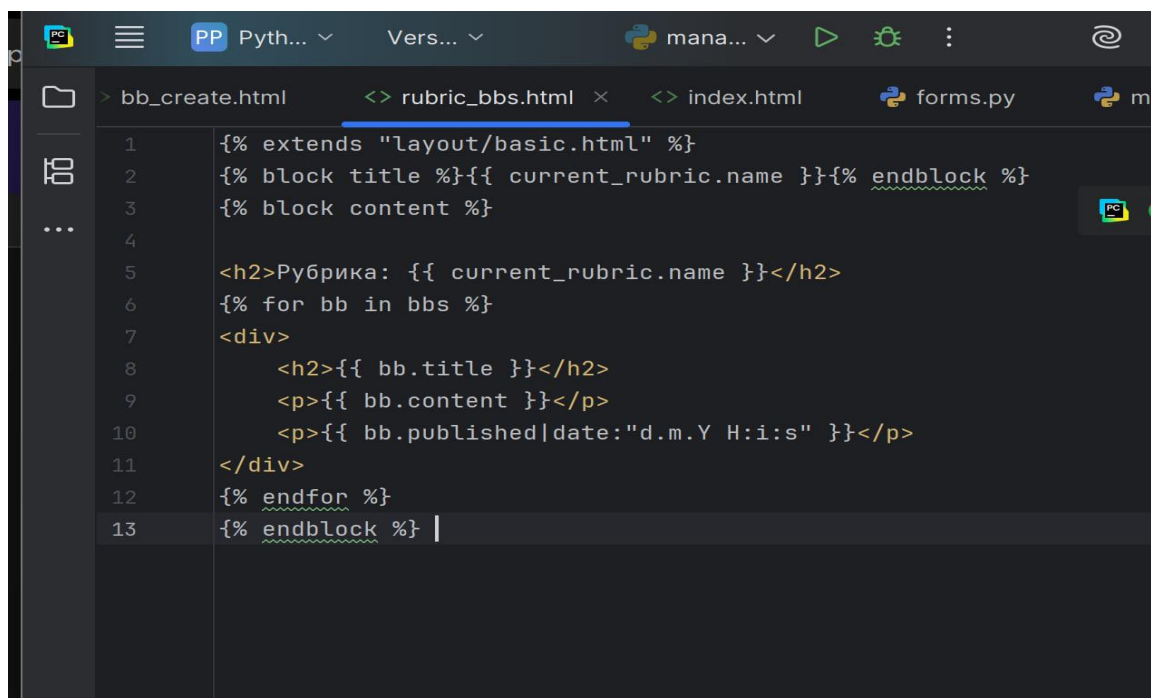
Сделали шаблон `bboard\index.html` производным от шаблона `layout/basic.html`.  
Новый код этого шаблона приведен в листинге 2.10



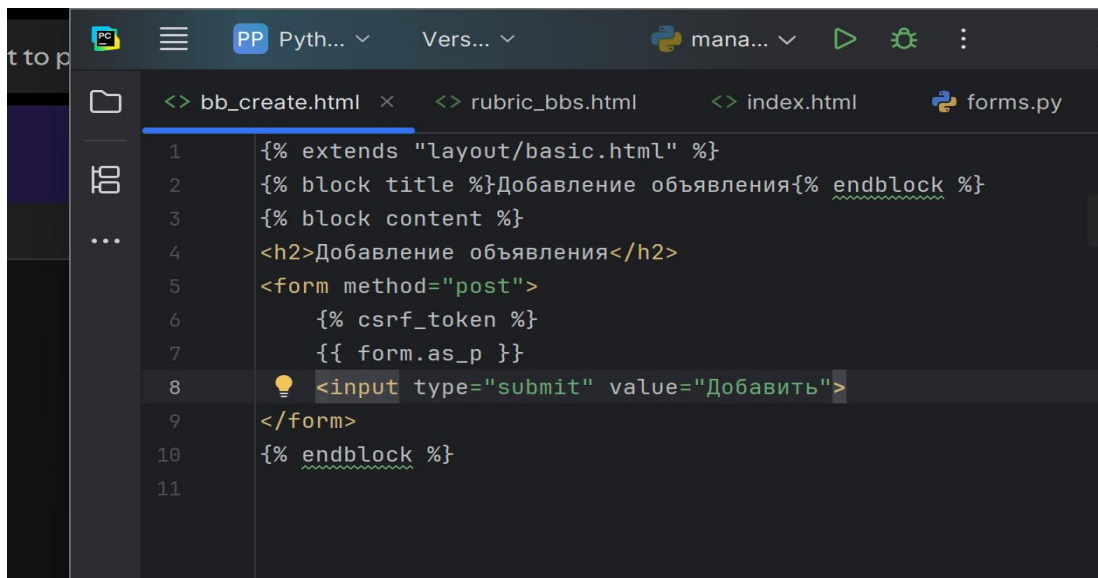
```
1 {% extends "layout/basic.html" %}
2 {% block content %}
3 {% for bb in bbs %}
4 <div>
5     <h2>{{ bb.title }}</h2>
6     <p>{{ bb.content }}</p>
7     <p><a href="{% url 'rubric_bbs' bb.rubric.pk %}">
8         {{ bb.rubric.name }}</a></p>
9     <p>{{ bb.published|date:"d.m.Y H:i:s" }}</p>
10 </div>
11 {% endfor %}
12 {% endblock %}
```

Аналогично исправили шаблоны `bboard\rubric_bbs.html` (листинг 2.11) и `bboard\bb_create.html` (листинг 2.12).

Сразу увидели, насколько уменьшился их объем.



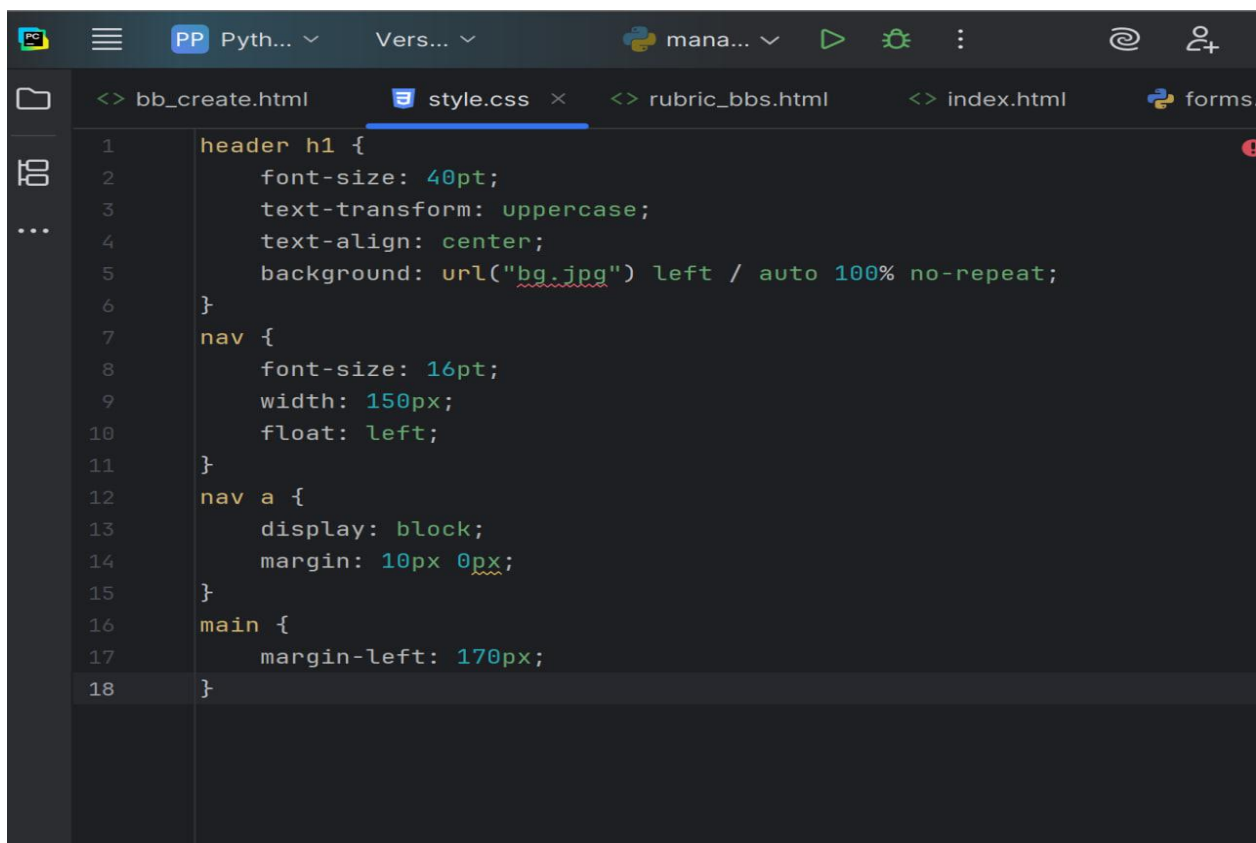
```
1 {% extends "layout/basic.html" %}
2 {% block title %}{{ current_rubric.name }}{% endblock %}
3 {% block content %}
4
5     <h2>Рубрика: {{ current_rubric.name }}</h2>
6     {% for bb in bbs %}
7     <div>
8         <h2>{{ bb.title }}</h2>
9         <p>{{ bb.content }}</p>
10        <p>{{ bb.published|date:"d.m.Y H:i:s" }}</p>
11    </div>
12    {% endfor %}
13    {% endblock %}
```



```
1 {% extends "layout/basic.html" %}
2 {% block title %}Добавление объявления{% endblock %}
3 {% block content %}
4 <h2>Добавление объявления</h2>
5 <form method="post">
6     {% csrf_token %}
7     {{ form.as_p }}
8     <input type="submit" value="Добавить">
9 </form>
10 {% endblock %}
11
```

## 2.8. Статические файлы

Наш первый сайт практически готов. Осталось навести небольшой лоск, применив таблицу стилей style.css из листинга 2.13



```
1 header h1 {
2     font-size: 40pt;
3     text-transform: uppercase;
4     text-align: center;
5     background: url("bg.jpg") left / auto 100% no-repeat;
6 }
7 nav {
8     font-size: 16pt;
9     width: 150px;
10    float: left;
11 }
12 nav a {
13     display: block;
14     margin: 10px 0px;
15 }
16 main {
17     margin-left: 170px;
18 }
```

Остановили отладочный веб-сервер.



Создали в папке пакета приложения bboard папку static, а в ней — вложенную папку bboard. В последней сохранили файлы style.css и bg.jpg. Открыли базовый шаблон layout/basic.html и вставили в него следующий код:

```
1      <!DOCTYPE html>
2      <html>
3          <head>
4              <meta charset="UTF-8">
5              <title>{% block title %}Главная{% endblock %} ::
6                  Доска объявлений</title>
7              <link rel="stylesheet" type="text/css"
8                  href="{% static 'bboard/style.css' %}">
9          </head>
10         <body>
11             <header>
12                 <h1>Объявления</h1>
13             </header>
14             <nav>
15                 <a href="{% url 'index' %}">Главная</a>
16                 <a href="{% url 'add' %}">Добавить</a>
17                 {% for rubric in rubrics %}
```

Главная веб-страница после применения к ней таблицы стилей:





[Главная](#)  
[Добавить](#)  
[Недвижимость](#)  
[Транспорт](#)

# ОБЪЯВЛЕНИЯ

## Добавление объявления

Товар:

Описание:

Цена:

Рубрика:

Добавить

### Вывод:

В ходе выполнения лабораторной работы были освоены принципы веб-разработки на базе фреймворка **Django**. Практическим итогом стало создание типового веб-ресурса — электронной доски объявлений. Реализация проекта осуществлялась исключительно штатными средствами Django, без привлечения сторонних библиотек, что позволило изучить базовые возможности инструмента.

### **Ссылка на репозиторий**

[https://github.com/SPOuBD25/LR2\\_Django.git](https://github.com/SPOuBD25/LR2_Django.git)